



Zusammenfassung

In diesem Kapitel haben wir Arrays als Möglichkeit diskutiert, um Sammlungsobjekte fester Größe zu erzeugen. Eindimensionale Arrays sind ein Sammlungstyp, der gut geeignet für solche Situationen ist, in denen die Anzahl der zu speichernden Elemente festgelegt und im Voraus bekannt ist. Sie sind auch syntaktisch ein wenig leichter als Sammlungen flexibler Größe wie **ArrayList**, insbesondere wenn sie zur Speicherung von Werten der primitiven Typen verwendet werden. Abgesehen von diesen speziellen Anwendungen bieten sie jedoch keine Funktionen, die nicht auch bereits mit Sammlungen wie Listen, Maps und Sets verfügbar wären.

NEUE BEGRIFFE IN DIESEM KAPITEL

Array, **for**-Schleife, Konditionaloperator, Zuordnungstabelle



Zusammenfassung der Konzepte

- **Array** Ein Array ist eine besondere Art von Sammlung, die eine festgelegte Anzahl von Elementen halten kann.
- **for-Schleife** Eine iterative Steuerungsstruktur, die häufig verwendet wird, wenn eine Indexvariable benötigt wird, um Folgeelemente aus einer Sammlung auszuwählen, wie eine **ArrayList** oder ein Array.

Übung 7.41 Schreiben Sie den folgenden Code neu, indem Sie die statische Methode **arraycopy** aus der Klasse **System** verwenden:

```
int[] kopie = new int[original.length];
for(int i = 0; i < original.length; i++) {
    kopie[i] = original[i];
}
```

Übung 7.42 Beschreiben Sie, was die folgenden statischen Methoden der Klasse **java.util.Arrays** machen: **asList**, **binarySearch**, **fill** und **sort**. Wenn es mehrere Versionen einer Methode gibt, verwenden Sie eine, die auf einem Integer-Array arbeitet.

Übung 7.43 Versuchen Sie, ein paar Beispielcodesegmente zu schreiben, mit dem Sie jede der Methoden aus der vorigen Übung benutzen können.

Übung 7.44 Schreiben Sie Anweisungen, um alle Elemente eines zweidimensionalen Arrays, **original**, in ein neues Array, **kopie**, zu kopieren. Sie sollten auch Anweisungen zur Erzeugung des **kopie**-Arrays schreiben.

Übung 7.45 Untersuchen Sie einige andere zweidimensionale zelluläre Automaten und implementieren Sie deren Regeln im *Brain*-Projekt.



KAPITEL

8

Klassenentwurf

Lernziele

Zentrale Konzepte in diesem Kapitel: Entwurf nach Zuständigkeiten, Kohäsion, Kopplung, Refactoring

Java-Konstrukte in diesem Kapitel: Aufzählungstypen, `switch`

In diesem Kapitel untersuchen wir die Faktoren, die den Entwurf einer Klasse beeinflussen. Wann ist ein Klassenentwurf gut, wann ist er schlecht? Ein guter Klassenentwurf kann kurzfristig mehr Aufwand bedeuten als ein schlechter, aber auf lange Sicht zahlt sich dieser Mehraufwand fast immer aus. Um gute Klassen zu schreiben, kann man sich an einige Entwurfsprinzipien halten. Insbesondere stellen wir hier die Sichtweise vor, dass ein Klassenentwurf sich an Zuständigkeiten orientieren sollte und dass Klassen ihre Daten kapseln sollten.

Dieses Kapitel orientiert sich, wie die vorherigen Kapitel auch, an einem Projekt. Sie können es einfach nur lesen und seiner Argumentation folgen, oder Sie können es sehr viel intensiver bearbeiten, indem Sie die Übungen am Projekt parallel zum Text bearbeiten.

Die Projektarbeit ist in drei Teile unterteilt. Im ersten Teil diskutieren wir einige notwendige Änderungen am Quelltext und entwickeln und zeigen komplette Lösungen zu den Übungen. Die Gesamtlösung für diesen Teil ist auch als ein eigenes Projekt im Begleitmaterial zu finden. Der zweite Teil schlägt zusätzliche Änderungen und Erweiterungen vor, die auf der Ebene des Klassenentwurfs diskutiert werden; die konkrete Umsetzung in der Programmierung überlassen wir jedoch dem Leser.

Der dritte Teil schlägt dann noch weitergehende Verbesserungen in Form von Übungen vor. Hier liefern wir keine Lösungen – die Übungen wenden das vorgestellte Material dieses Kapitels an.

Die Implementierung aller Teile ist ein gutes Programmierprojekt, das über mehrere Wochen laufen kann. Sie kann auch sehr gut in Gruppenarbeit vorgenommen werden.

8.1 Einführung

Eine Anwendung kann implementiert werden und ihre Aufgabe erfüllen, obwohl ihr Klassenentwurf schlecht ist. Allein durch das Ausführen einer Anwendung kann man üblicherweise nicht darauf schließen, ob ihr Quelltext gut oder schlecht strukturiert ist.

Die Probleme treten meist dann auf, wenn ein Wartungsprogrammierer Änderungen an einer bestehenden Anwendung vornehmen muss. Wenn ein Programmierer beispielsweise einen Fehler beheben oder neue Funktionalität hinzufügen will, dann kann diese Aufgabe bei gut entworfenen Klassen sehr einfach und gradlinig umsetzbar sein, während sie bei einem schlechten Klassenentwurf viel Zeit und Energie kosten kann.

Bei großen Anwendungen tritt dieser Effekt schon während der Erstimplementierung auf. Wenn eine Implementierung schon mit einem schlechten Entwurf beginnt, dann kann die Arbeit an ihr schließlich sehr komplex werden, und die gesamte Anwendung wird dann entweder nicht fertig, enthält Fehler oder braucht sehr viel länger. In der Praxis warten, erweitern und verkaufen Unternehmen eine Anwendung oft über viele Jahre. Es ist heutzutage keine Seltenheit, dass die Implementierung einer Software, die wir im Laden kaufen können, bereits vor 10 Jahren begonnen wurde. In einer solchen Situation kann sich eine Softwarefirma schlecht strukturierten Quelltext schlicht nicht leisten.

Da die meisten Effekte von schlechtem Klassenentwurf am offensichtlichsten werden, wenn man eine Anwendung anpassen oder erweitern will, werden wir genau das tun. In diesem Kapitel verwenden wir *Die Welt von Zuul*, eine rudimentäre Implementierung eines textbasierten Adventure-Games. In seiner vorgegebenen Form ist es nicht sonderlich spannend: Es ist insbesondere noch unvollständig. Am Ende dieses Kapitels werden Sie jedoch so weit sein, dass Sie Ihre Phantasie einsetzen und Ihre eigene Version des Spiels entwerfen und implementieren können, die spannender und interessanter ist.

Die Welt von Zuul

Unsere Version des Spiels basiert auf einem Spiel namens *Adventure*, das in den frühen Siebzigern von Will Crowther entwickelt und von Don Woods erweitert wurde. Das Original ist auch unter dem Namen *Colossal Cave Adventure* bekannt. Für seine Zeit war es ein wunderbar kreatives und aufwendig gestaltetes Spiel, in dem man seinen Weg durch ein komplexes System von Gewölben finden musste, dabei versteckte Schätze heben, geheime Parolen anwenden und andere spannende Dinge tun musste, um letztlich die maximale Punktzahl zu erzielen. Sie können mehr über das Spiel erfahren unter <http://jerz.setonhill.edu/lif/canon/Adventure.htm> und unter <http://www.rickadams.org/adventure/>, oder führen Sie eine Websuche nach „Colossal Cave Adventure“ durch.

Während wir die vorgegebene Anwendung erweitern, werden wir einige Aspekte ihres Klassenentwurfs ansprechen. Wir werden feststellen, dass die Implementierung, auf der wir aufsetzen, einige schlechte Entwurfsentscheidungen enthält, und wir werden untersuchen, inwieweit diese unsere Aufgabe beeinflussen und wie wir sie beheben können.

Im Begleitmaterial zu diesem Buch werden Sie zwei Versionen des *Zuul*-Projekts finden: *Zuul-schlecht* und *Zuul-besser*. Beide implementieren die gleiche Funktionalität, aber sie unterscheiden sich an einigen Punkten in der Klassenstruktur. Die eine Version ist schlecht entworfen, die andere besser. Die Tatsache, dass wir dieselbe Funktionalität entweder gut oder schlecht strukturiert umsetzen können, verdeutlicht, dass ein schlechter Entwurf üblicherweise nicht durch die Komplexität des zu lösenden Problems entschuldigt werden kann. Ein schlechter Entwurf hängt von den Entscheidungen ab, die wir bei der Lösung eines Problems treffen. Wir können nicht behaupten, dass es keinen anderen Weg zur Lösung des Problems gegeben hat, um einen schlechten Entwurf zu rechtfertigen.

Deshalb werden wir das Projekt mit dem schlechten Entwurf untersuchen und aufzeigen, warum er schlecht ist, um ihn dann anschließend zu verbessern. In der besseren Version des Projekts sind die Änderungen implementiert, die wir nun diskutieren werden.

Übung 8.1 Öffnen Sie das Projekt *Zuul-schlecht*. (Dieses Projekt heißt „schlecht“, weil es einige schlechte Entwurfsentscheidungen beinhaltet, und wir wollen keinen Zweifel daran lassen, dass dieses Projekt nicht als ein Beispiel für gute Programmierpraxis genutzt werden soll.) Starten Sie die Anwendung und erkunden Sie diese. Im Projektkommentar finden Sie Hinweise zum Starten.

Während Sie die Anwendung erkunden, beantworten Sie folgende Fragen:

- Was tut diese Anwendung?
- Welche Befehle akzeptiert das Spiel?
- Was bewirken die einzelnen Befehle?
- Wie viele Räume gibt es in der virtuellen Umgebung?
- Zeichnen Sie eine Karte der gegebenen Räume.

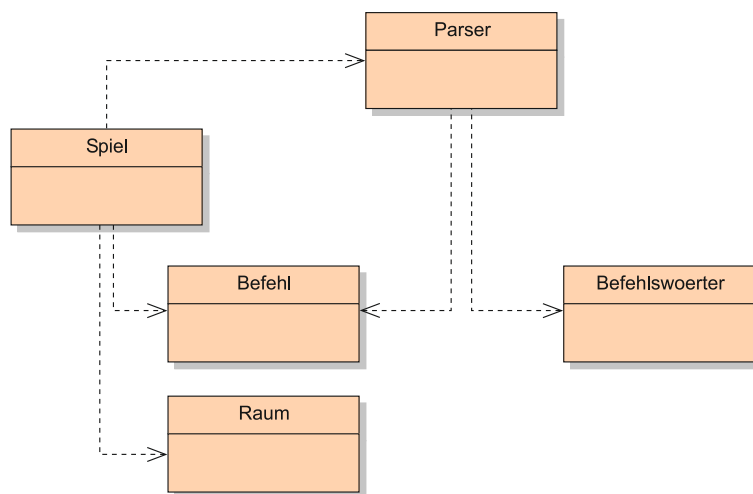
Übung 8.2 Nachdem Sie herausgefunden haben, was die Anwendung insgesamt tut, sollten Sie nun die individuellen Klassen erkunden. Schreiben Sie für jede Klasse ihren Einsatzzweck auf. Dazu müssen Sie sich den Quelltext der Klassen ansehen. Beachten Sie, dass Sie möglicherweise nicht jedes Detail verstehen werden (und auch nicht müssen). Häufig reicht es, die Kommentare zu lesen und die Methodenköpfe zu untersuchen.

8.2 Die Welt von Zuul

Durch **Übung 8.1** werden Sie festgestellt haben, dass das Spiel nicht sonderlich abenteuerlich ist. Tatsächlich ist es in diesem Zustand ziemlich langweilig. Aber es bietet uns eine gute Basis, um ein Spiel zu entwerfen und zu implementieren, das hoffentlich etwas interessanter sein wird.

Wir untersuchen zuerst die gegebenen Klassen und versuchen ihre Funktionsweise herauszufinden. Das Klassendiagramm ist in Abbildung 8.1 zu sehen.

Abbildung 8.1
Das Klassendiagramm
für *Zuul*.



Das Diagramm zeigt fünf Klassen: **Parser**, **Befehlswoerter**, **Befehl**, **Raum** und **Spiel**. Eine Untersuchung des Quelltextes zeigt, dass diese Klassen recht gut dokumentiert sind und wir schon einen guten Überblick über die Anwendung bekommen können, indem wir nur die Klassenkommentare lesen. (Dies zeigt uns auch, dass schlechter Entwurf etwas Grundlegenderes ist, das nicht am Aussehen oder an der Dokumentation der Klassen erkennbar ist.) Wir verstehen das besser, wenn wir im Quelltext untersuchen, welche Methoden die einzelnen Klassen haben und was diese Methoden tun. Wir fassen den Zweck der einzelnen Klassen hier kurz zusammen:

- **Befehlswoerter**: Die Klasse **Befehlswoerter** definiert die gültigen Befehle für das Spiel. Sie hält dazu ein Array von **string**-Objekten für die Befehlskörper.
- **Parser**: Der Parser liest Eingabezeilen von der Konsole und versucht, sie als Befehle zu interpretieren. Er erzeugt Objekte der Klasse **Befehl**, die die eingegebenen Befehle repräsentieren.
- **Befehl**: Ein **Befehl**-Objekt repräsentiert einen Befehl, den der Benutzer eingegeben hat. Es bietet Methoden, mit denen wir leicht herausfinden können, ob es ein gültiger Befehl ist und aus welchen Wörtern sich der Befehl zusammensetzt.
- **Raum**: Ein **Raum**-Objekt repräsentiert einen Ort im Spielgeschehen. Räume können Ausgänge haben, die zu anderen Räumen führen.
- **Spiel**: Die Klasse **Spiel** ist die Hauptklasse des Spiels. Sie initialisiert das Spiel und liest dann in einer Schleife Befehle ein und führt sie aus. Die Klasse enthält auch die Implementierung der einzelnen Befehle.

Übung 8.3 Entwerfen Sie ein eigenes Szenario für ein Spiel. Tun Sie das ohne Computer. Denken Sie dabei nicht über Implementierungen, Klassen oder überhaupt Programmieren im Allgemeinen nach. Erfinden Sie einfach ein spannendes Spiel. Sie können dies auch in Gruppenarbeit tun.

Zugelassen ist alles, bei dem ein Spieler sich durch unterschiedliche Räumlichkeiten bewegt. Hier ein paar Anregungen:

- Sie sind ein weißes Blutkörperchen, das durch einen Körper fließt und auf der Suche nach angreifenden Viren ist ...
- Sie haben sich in einem Einkaufszentrum verlaufen und müssen den Ausgang finden ...
- Sie sind ein Maulwurf in seiner Höhle und erinnern sich nicht, an welcher Stelle Sie Ihre Futterreserven für den Winter versteckt haben ...
- Sie sind ein Abenteurer, der ein Verließ voller Monster und anderer Kreaturen durchsucht ...
- Sie gehören zu einem Sprengstoffkommando und müssen eine Bombe finden und entschärfen, bevor sie explodiert ...

Stellen Sie sicher, dass Ihr Spiel ein Ziel hat (sodass es ein Ende gibt und der Spieler „gewinnen“ kann). Integrieren Sie viele Elemente, die das Spiel interessant machen können (Falltüren; magische Gegenstände; Charaktere, die nur helfen, wenn sie gefüttert werden; Zeitbeschränkungen – alles, was Ihnen in den Sinn kommt). Lassen Sie Ihrer Phantasie freien Lauf.

Sorgen Sie sich an diesem Punkt nicht darum, wie diese Dinge implementiert werden könnten.

8.3 Kopplung und Kohäsion

Um die Aussage zu stützen, dass manche Entwürfe besser sind als andere, müssen wir einige Begriffe definieren, mit deren Hilfe wir die wichtigen Eigenschaften von Klassenentwürfen diskutieren können. Zwei Begriffe spielen eine zentrale Rolle bei der Qualität von Klassenentwürfen: *Kopplung* und *Kohäsion*.

Der Begriff *Kopplung* bezieht sich auf die Art, wie Klassen miteinander verknüpft sind. Wir haben bereits in früheren Kapiteln betont, dass wir Anwendungen als Mengen von kooperierenden Klassen entwerfen wollen, die über wohldefinierte Schnittstellen miteinander kommunizieren. Der Grad der Kopplung gibt an, wie eng diese Klassen miteinander verknüpft sind. Wir streben einen möglichst niedrigen Grad an Kopplung an, eine möglichst *lose Kopplung*.

Der Grad der Kopplung bestimmt, wie schwierig Änderungen an einer Anwendung sind. In einer eng gekoppelten Klassenstruktur kann eine Änderung an einer Klasse viele Änderungen an anderen Klassen nach sich ziehen. Das ist genau der Effekt, den wir vermeiden wollen, denn auf diese Weise zieht sich eine Änderung schnell durch eine ganze Anwendung. Zusätzlich kann es schwierig und zeitaufwendig sein, alle betroffenen Stellen zu finden und anzupassen.

Konzept

Der Begriff **Kopplung** beschreibt den Grad der Abhängigkeiten zwischen Klassen. Wir streben für ein System eine möglichst lose Kopplung an – also ein System, in dem jede Klasse weitgehend unabhängig ist und mit anderen Klassen nur über möglichst schmale, wohldefinierte Schnittstellen kommuniziert.

In einem lose gekoppelten System hingegen können wir häufig Änderungen an einer einzelnen Klasse vornehmen, ohne dass irgendeine andere Klasse geändert werden muss, und die Anwendung läuft weiterhin. Wir werden einige konkrete Beispiele von loser und enger Kopplung in diesem Kapitel zeigen.

Konzept

Der Begriff **Kohäsion** beschreibt, wie gut eine Programmeinheit eine logische Aufgabe oder Einheit abbildet. In einem System mit hoher Kohäsion ist jede Programmeinheit (eine Methode, eine Klasse oder ein Modul) verantwortlich für genau eine wohldefinierte Aufgabe oder Einheit. Ein guter Klassenentwurf weist einen hohen Grad an Kohäsion auf.

Der Begriff *Kohäsion* bezieht sich auf die Anzahl und Vielfalt der Aufgaben, für die eine einzelne Einheit in einer Anwendung zuständig ist. Kohäsion ist wichtig für einzelne Klassen und auch für einzelne Methoden.¹

Idealerweise sollte eine Programmeinheit für genau eine in sich geschlossene Aufgabe zuständig sein (also für eine Aufgabe, die als eine zusammenhängende, logische Einheit angesehen werden kann). Eine Methode sollte eine logische Operation implementieren und eine Klasse sollte genau einen Typ von Objekt modellieren. Der Hauptanlass für Kohäsion ist die Wiederverwendung: Wenn eine Methode oder eine Klasse für genau eine sehr klar definierte Aufgabe verantwortlich ist, dann ist die Wahrscheinlichkeit höher, dass diese Einheit auch in anderen Zusammenhängen eingesetzt werden kann. Ein ergänzender Vorteil beim Befolgen dieses Prinzips ist, dass im Falle von Änderungen die Stellen, die von diesen Änderungen betroffen sind, eher in einer Einheit zu finden sind.

Wir werden nun anhand einiger Beispiele zeigen, wie Kohäsion die Qualität eines Klassenentwurfs beeinflussen kann.

Übung 8.4 Zeichnen Sie (auf Papier) eine Karte für das Spiel, das Sie in **Übung 8.3** erfunden haben. Öffnen Sie das Projekt *Zuul-schlecht* und speichern Sie es unter einem anderen Namen (*Zuul* beispielsweise). An diesem Projekt werden Sie im Laufe dieses Kapitels Veränderungen und Erweiterungen vornehmen. Sie können das Suffix *-schlecht* weglassen, da dieses Projekt (hoffentlich) bald nicht mehr ganz so schlecht ist.

Als ersten Schritt sollen Sie die Methode `raeumeAnlegen` so ändern, dass sie die Räume und Ausgänge für Ihr Spiel anlegt.

8.4 Code-Duplizierung

Die Duplizierung von Quelltextabschnitten (Code-Duplizierung) ist ein Indiz schlechten Entwurfs. Die Klasse `Spiel` in Listing 8.1 enthält ein Beispiel für eine solche Duplizierung. Das Problem mit Code-Duplizierung ist, dass Änderungen an der einen Stelle immer auch an der anderen durchgeführt werden müssen, damit keine Inkonsistenzen entstehen. Dies erhöht den Aufwand für einen Wartungsprogrammierer und die Gefahr von Fehlern. Es kommt häufig vor, dass ein Wartungsprogrammierer die eine Kopie des Abschnitts findet, den Fehler behebt und dann glaubt, alles sei erledigt. Es gibt keinen Hinweis darauf, dass eine zweite Kopie des Abschnitts existiert, und diese kann weiterhin im falschen Zustand verbleiben.

Konzept

Code-Duplizierung (ein Quelltextabschnitt erscheint mehr als einmal in einer Anwendung) ist ein Indiz für einen schlechten Entwurf. Sie sollte vermieden werden.

¹ Wir verwenden teilweise auch den Begriff *Modul* (oder *Paket* in Java), um uns auf Einheiten zu beziehen, die aus mehreren Klassen bestehen. *Kohäsion* ist auch auf dieser Ebene wichtig.

```

public class Spiel
{
    // ... Abschnitte hier ausgelassen ...

    private void raumeAnlegen()
    {
        Raum draussen, hoersaal, cafeteria, labor, buero;

        // die Räume erzeugen
        draussen = new Raum("vor dem Haupteingang der Universität");
        hoersaal = new Raum("in einem Vorlesungssaal");
        cafeteria = new Raum("in der Cafeteria der Uni");
        labor = new Raum("in einem Rechnerraum");
        buero = new Raum("im Verwaltungsbüro der Informatik");

        // die Ausgänge initialisieren
        draussen.setzeAusgaenge(null, hoersaal, labor, cafeteria);
        hoersaal.setzeAusgaenge(null, null, null, draussen);
        cafeteria.setzeAusgaenge(null, draussen, null, null);
        labor.setzeAusgaenge(draussen, buero, null, null);
        buero.setzeAusgaenge(null, null, null, labor);

        aktuellerRaum = draussen; // das Spiel startet draussen
    }

    // ... Abschnitte hier ausgelassen ...

    /**
     * Einen Begrüßungstext für den Spieler ausgeben.
     */
    private void willkommenTextAusgeben()
    {
        System.out.println();
        System.out.println("Willkommen zu Zuul!");
        System.out.println("Zuul ist ein neues, unglaublich langweiliges Spiel.");
        System.out.println("Tippen Sie 'help', wenn Sie Hilfe brauchen.");
        System.out.println();
        System.out.println("Sie sind " + aktuellerRaum.gibBeschreibung());
        System.out.print("Ausgänge: ");
        if(aktuellerRaum.nordausgang != null) {
            System.out.print("north ");
        }
        if(aktuellerRaum.ostausgang != null) {
            System.out.print("east ");
        }
        if(aktuellerRaum.suedausgang != null) {
            System.out.print("south ");
        }
        if(aktuellerRaum.westausgang != null) {
            System.out.print("west ");
        }
        System.out.println();
    }

    // ... Abschnitte hier ausgelassen ...

    /**
     * Versuche, in eine Richtung zu gehen. Wenn es einen Ausgang gibt,
     * wechsele in den neuen Raum, ansonsten gib eine Fehlermeldung
     * aus.
     */
    private void wechseleRaum(Befehl befehl)
    {
        if(!befehl.hatZweitesWort()) {
            // Gibt es kein zweites Wort, wissen wir nicht, wohin ...
            System.out.println("Wohin möchten Sie gehen?");
            return;
        }

        String richtung = befehl.gibZweitesWort();

        // Wir versuchen, den Raum zu verlassen.
        Raum naechsterRaum = null;
        if(richtung.equals("north")) {
            naechsterRaum = aktuellerRaum.nordausgang;
        }
        if(richtung.equals("east")) {
            naechsterRaum = aktuellerRaum.ostausgang;
        }
    }
}

```

Listing 8.1

Ausgewählte Abschnitte aus der (schlecht entworfenen) Klasse `Spiel`.

```

        if(richtung.equals("south")) {
            naechsterRaum = aktuellerRaum.suedausgang;
        }
        if(richtung.equals("west")) {
            naechsterRaum = aktuellerRaum.westausgang;
        }

        if (naechsterRaum == null) {
            System.out.println("Dort ist keine Tür!");
        }
        else {
            aktuellerRaum = naechsterRaum;
            System.out.println("Sie sind " + aktuellerRaum.gibBeschreibung());
            System.out.print("Ausgänge: ");
            if(aktuellerRaum.nordausgang != null) {
                System.out.print("north ");
            }
            if(aktuellerRaum.ostausgang != null) {
                System.out.print("east ");
            }
            if(aktuellerRaum.suedausgang != null) {
                System.out.print("south ");
            }
            if(aktuellerRaum.westausgang != null) {
                System.out.print("west ");
            }
            System.out.println();
        }
    }

    // ... Abschnitte hier ausgelassen ...
}

```

Sowohl die Methode **willkommenstextAusgeben** als auch die Methode **wechsleRaum** enthalten den folgenden Quelltextabschnitt:

```

        System.out.println("Sie sind " + aktuellerRaum.gibBeschreibung());
        System.out.print("Ausgänge: ");
        if(aktuellerRaum.nordausgang != null) {
            System.out.print("north ");
        }
        if(aktuellerRaum.ostausgang != null) {
            System.out.print("east ");
        }
        if(aktuellerRaum.suedausgang != null) {
            System.out.print("south ");
        }
        if(aktuellerRaum.westausgang != null) {
            System.out.print("west ");
        }
        System.out.println();
    }

```

Code-Duplizierung ist ein Zeichen für schlechte Kohäsion. Die Wurzel des Problems liegt hier darin, dass beide Methoden zweierlei Dinge tun: **willkommenstextAusgeben** gibt einen Willkommenstext aus und liefert Informationen über den aktuellen Raum, während **wechsleRaum** den Raum wechselt und dann Informationen über den (neuen) aktuellen Raum ausgibt.

Beide Methoden geben Informationen über den aktuellen Raum aus, aber keine kann die andere aufrufen, weil sie auch noch andere Dinge erledigen. Das ist ein schlechter Entwurf.

Ein besserer Entwurf würde eine separate Methode mit verbesserter Kohäsion benutzen, deren einzige Aufgabe ist, Informationen über den aktuellen Raum auszugeben (Listing 8.2). Sowohl `willkommenstextAusgeben` als auch `wechsleRaum` können dann diese Methode aufrufen, wenn sie diese Informationen ausgeben lassen wollen. So wird vermieden, dass der gleiche Text zweimal geschrieben werden muss, und wir brauchen bei einer Änderung nur eine Stelle zu korrigieren.

```
private void rauminfoAusgeben()
{
    System.out.println("Sie sind " + aktuellerRaum.gibBeschreibung());
    System.out.println("Ausgänge: ");
    if(aktuellerRaum.nordausgang != null) {
        System.out.println("north ");
    }
    if(aktuellerRaum.ostausgang != null) {
        System.out.println("east ");
    }
    if(aktuellerRaum.suedausgang != null) {
        System.out.println("south ");
    }
    if(aktuellerRaum.westauegang != null) {
        System.out.println("west ");
    }
    System.out.println();
}
```

Listing 8.2
`rauminfoAusgeben`
als eigene Methode.

Übung 8.5 Implementieren und benutzen Sie eine eigene Methode `rauminfoAusgeben` in Ihrem Projekt, so wie oben beschrieben. Testen Sie Ihre Änderungen.

8.5 Erweiterungen für Zuul

Das Projekt *Zuul-schlecht* funktioniert. Wir können es ausführen und es macht all das, was es tun soll. Aber an einigen Stellen ist es schlecht entworfen. Eine gut entworfene Alternative würde genauso funktionieren. Indem wir lediglich das Programm ausführen, würden wir keinen Unterschied feststellen.

Sobald wir aber Änderungen am Projekt vornehmen wollen, stellen wir fest, dass es erheblich aufwendiger ist, ein schlecht entworfenes System zu ändern als ein gut entworfenes System. Wir werden dies verdeutlichen, indem wir einige Änderungen am Projekt vornehmen. Während wir dies tun, werden wir immer wieder auf schlechte Entwurfsentscheidungen im bestehenden Quelltext hinweisen und sie beheben, bevor wir die Änderungen vornehmen.

8.5.1 Die Aufgabe

Als Erstes werden wir versuchen, neue Bewegungsrichtungen in das Spiel einzubauen. Momentan kann sich ein Spieler in vier Richtungen bewegen: *north*, *east*, *south* und *west*. Wir wollen nun auch mehrstöckige Gebäude ermöglichen (oder Keller oder Verliese oder was Sie Ihrem Spiel sonst noch hinzufügen möchten) und deshalb *up* und *down* als mögliche Richtungen einführen.

8.5.2 Ermitteln der betroffenen Quelltextstellen

Eine Untersuchung der gegebenen Klassen zeigt uns, dass mindestens zwei Klassen von dieser Änderungen betroffen sind: **Spiel** und **Raum**.

Raum ist die Klasse, die (neben anderen Dingen) die Ausgänge eines Raums speichert, und in **Spiel** werden, wie wir in Listing 8.1 gesehen haben, diese Informationen benutzt, um dem Benutzer mögliche Ausgänge melden zu können und um den Raum zu wechseln.

Die Klasse **Raum** ist recht kurz. Ihr Quelltext wird in Listing 8.3 gezeigt. Beim Lesen der Klassendefinition stellen wir fest, dass die Ausgänge an zwei Stellen erwähnt werden: Sie werden zu Anfang als Datenfelder aufgezählt und dann in der Methode **setzeAusgänge** gesetzt. Um zwei neue Richtungen (etwa **deckenausgang** und **boden- ausgang**) einzufügen, müssten wir sie an diesen Stellen einführen.

Listing 8.3

Quelltext der
(schlecht entworfenen)
Klasse **Raum**.

```
public class Raum
{
    public String beschreibung;
    public Raum nordausgang;
    public Raum suedausgang;
    public Raum ostdausgang;
    public Raum westausgang;

    /**
     * Erzeuge einen Raum mit einer Beschreibung. Ein Raum
     * hat anfangs keine Ausgänge. Eine Beschreibung hat die Form
     * "in einer Küche" oder "auf einem Sportplatz".
     * @param beschreibung die Beschreibung des Raums
     */
    public Raum(String beschreibung)
    {
        this.beschreibung = beschreibung;
    }

    /**
     * Definiere die Ausgänge dieses Raums. Jede Richtung
     * führt entweder in einen anderen Raum oder ist 'null'
     * (kein Ausgang).
     * @param norden der Nordausgang
     * @param osten der Ostdausgang
     * @param sueden der Südausgang
     * @param westen der Westausgang
     */
    public void setzeAusgaenge(Raum norden, Raum osten,
                               Raum sueden, Raum westen)
    {
        if(norden != null) {
            nordausgang = norden;
        }
        if(osten != null) {
            ostdausgang = osten;
        }
        if(sueden != null) {
            suedausgang = sueden;
        }
        if(westen != null) {
            westausgang = westen;
        }
    }

    /**
     * @return die Beschreibung dieses Raums
     */
    public String gibBeschreibung()
    {
        return beschreibung;
    }
}
```


Es ist etwas aufwendiger, die betroffenen Stellen in der Klasse **Spiel** zu finden. Der Quelltext ist etwas länger (er ist hier nicht vollständig aufgeführt) und das Finden der betroffenen Stellen erfordert einige Geduld und Vorsicht.

Wenn wir den Quelltext in Listing 8.1 betrachten, dann sehen wir, dass sich die Klasse **Spiel** sehr stark auf die Informationen über die Ausgänge bezieht. Ein **Spiel**-Objekt hält eine Referenz auf einen Raum in der Variablen **aktuellerRaum** und greift häufig auf die Ausgangsinformationen dieses Raums zu:

- In der Methode **raeumeAnlegen** werden die Ausgänge definiert.
- In der Methode **willkommenstextAusgeben** werden die Ausgänge des aktuellen Raums ausgegeben, damit ein Spieler beim Spielstart weiß, in welche Richtungen er gehen kann.
- In der Methode **wechsleRaum** werden die Ausgänge benutzt, um den nächsten Raum zu finden. Sie werden dann noch einmal benutzt, um die Ausgänge des Raums auszugeben, den wir gerade betreten haben.

Wenn wir nun die beiden neuen Ausgangsrichtungen *up* und *down* hinzufügen wollen, dann müssen wir die Option an all diesen Stellen einbauen. Bevor Sie dies tun, sollten Sie jedoch den nächsten Abschnitt lesen.

8.6 Kopplung

Der Umstand, dass alle Ausgänge an so vielen Stellen im Quelltext aufgezählt werden, ist symptomatisch für einen schlechten Klassenentwurf. Bei der Deklaration der Variablen für die Ausgänge müssen wir eine Variable pro Ausgang definieren; in der Methode **setzeAusgaenge** gibt es eine **if**-Anweisung für jeden Ausgang; in der Methode **wechsleRaum** gibt es eine **if**-Anweisung für jeden Ausgang; in der Methode **rauminfoAusgeben** gibt es eine **if**-Anweisung für jeden Ausgang; und so weiter. Diese Entwurfsentscheidung bürdet uns nun einiges an Arbeit auf: Beim Einführen neuer Ausgänge müssen wir alle Stellen finden und diese beiden neuen Fälle einfügen. Stellen Sie sich den Aufwand vor, wenn wir uns entschieden hätten, Richtungen wie *northwest*, *southwest* etc. einzuführen!

Um diese Situation zu verbessern, entscheiden wir uns statt einzelner Variablen für eine **HashMap** zur Speicherung der Ausgänge. Auf diese Weise sollten wir Quelltext schreiben können, der mit einer beliebigen Anzahl von Ausgängen umgehen kann und nicht so viele Änderungen nach sich zieht. Die **HashMap** enthält eine Abbildung eines Richtungsnamens (etwa "**north**") auf den Raum, der in dieser Richtung liegt (ein **Raum**-Objekt). Jeder Eintrag besteht somit aus einer Zeichenkette als Schlüssel und einem **Raum**-Objekt als Wert.

Dies ändert die Art, wie ein Raum Informationen über seine Nachbarräume hält. Theoretisch sollte eine solche Änderung nur die *Implementierung* der Klasse betreffen (wie die Informationen über die Ausgänge gespeichert werden), aber nicht ihre *Schnittstelle* (was ein Raum speichert).

Idealerweise sollten andere Klassen nicht durch eine Änderung an der Implementierung in einer Klasse betroffen sein. Dann hätten wir wirklich eine *lose* Kopplung.

In unserem Beispiel klappt das nicht. Wenn wir die Datenfelder in der Klasse **Raum** entfernen und durch eine **HashMap** ersetzen, lässt sich die Klasse **Spiel** nicht mehr übersetzen. Sie nimmt an vielen Stellen Bezug auf die Datenfelder für die Ausgänge in der Klasse **Raum**, die folglich alle Fehler produzieren würden.

Wir sehen, dass wir hier einen Fall von *enger* Kopplung haben. Um dies zu bereinigen, werden wir vor der Einführung der **HashMap** die beiden Klassen entkoppeln.

8.6.1 Kapselung zur Reduzierung der Kopplung

Eines der Hauptprobleme in diesem Beispiel ist die Benutzung von öffentlichen Datenfeldern. Die Datenfelder für die Ausgänge sind in der Klasse **Raum** alle als **public** deklariert. Diese Klasse wurde also offenbar von einer Person erstellt, die sich nicht an eine bereits vorgestellte Grundregel gehalten hat („Definieren Sie Datenfelder niemals öffentlich!“). Wir werden nun sehen, was das für Folgen haben kann. Die Klasse **Spiel** kann direkt auf diese Felder zugreifen (und tut dies auch ausführlich). Dadurch, dass die Klasse **Raum** ihre Datenfelder öffentlich anbietet, hat sie nicht nur den Umstand offengelegt, dass sie über Ausgänge verfügt; sie hat außerdem auch offengelegt, wie diese Ausgänge genau implementiert sind. Dies bricht mit einem fundamentalen Prinzip für guten Klassenentwurf: der *Kapselung*.

Konzept

Saubere **Kapselung** reduziert die Kopplung und führt deshalb zu besseren Entwürfen.

Die Richtlinie für Kapselung (Informationen über die Implementierung verbergen) besagt, dass nur Informationen über das *Was* einer Klasse (was sie leistet) nach außen sichtbar sein sollten, nicht aber das *Wie* (ihre Realisierung). Das hat einen entscheidenden Vorteil: Wenn keiner anderen Klasse bekannt ist, wie wir unsere Implementierung vornehmen, dann können wir die Implementierung ändern, ohne dass andere Klassen beeinträchtigt werden.

Wir können diese Trennung zwischen dem *Was* und dem *Wie* erzwingen, indem wir die Datenfelder privat deklarieren und eine sondierende Methode für den Zugriff auf sie definieren. Die erste Stufe unserer geänderten Klasse **Raum** ist in Listing 8.4 gezeigt.

Listing 8.4

Eine sondierende Methode, um die Kopplung zu reduzieren.

```
public class Raum
{
    public String beschreibung;
    public Raum nordausgang;
    public Raum suedausgang;
    public Raum ostausgang;
    public Raum westausgang;

    // andere Methoden unverändert

    public Raum gibAusgang(String richtung)
    {
        if(richtung.equals("north")) {
            return nordausgang;
        }
        if(richtung.equals("east")) {
            return ostausgang;
        }
        if(richtung.equals("south")) {
            return suedausgang;
        }
        if(richtung.equals("west")) {
            return westausgang;
        }
        return null;
    }
}
```

Nach dieser Änderung an der Klasse **Raum** müssen wir auch die Klasse **Spiel** anpassen. An allen Stellen, an denen bisher direkt auf die Datenfelder zugegriffen wurde, verwenden wir nun die sondierende Methode. Beispielsweise schreiben wir statt

```
naechsterRaum = aktuellerRaum.ostausgang;
```

nun

```
naechsterRaum = aktuellerRaum.gibAusgang("east");
```

Dies macht auch einen Abschnitt in der Klasse **Spiel** viel einfacher. In der Methode **wechsleRaum** führt diese Änderung zu folgendem Quelltext:

```
Raum naechsterRaum = null;
if (richtung.equals("north")) {
    naechsterRaum = aktuellerRaum.gibAusgang("north");
}
if (richtung.equals("east")) {
    naechsterRaum = aktuellerRaum.gibAusgang("east");
}
if (richtung.equals("south")) {
    naechsterRaum = aktuellerRaum.gibAusgang("south");
}
if (richtung.equals("west")) {
    naechsterRaum = aktuellerRaum.gibAusgang("west");
}
```

Dieser lange Abschnitt kann nun durch die folgende Anweisung ersetzt werden:

```
Raum naechsterRaum = aktuellerRaum.gibAusgang(richtung);
```

Übung 8.6 Führen Sie die beschriebenen Änderungen an den Klassen **Raum** und **Spiel** durch.

Übung 8.7 Führen Sie eine ähnliche Änderung auch an der Methode **raum-infoAusgeben** in der Klasse **Spiel** durch, sodass die Details der Ausgänge nun von **Raum** und nicht mehr von **Spiel** geliefert werden. Definieren Sie dazu eine Methode in der Klasse **Raum** mit folgendem Kopf:

```
/**
 * Liefere eine Beschreibung der Ausgänge dieses Raumes,
 * beispielsweise "Ausgänge: north west".
 * @return eine Beschreibung der verfügbaren Ausgänge.
 */
public String gibAusgaengeAlsString()
```

Bisher haben wir noch nicht die Repräsentation der Ausgänge in der Klasse **Raum** geändert. Wir haben lediglich die Schnittstelle etwas aufgeräumt. Die *Änderung* in der Klasse **Spiel** ist minimal – statt dem Zugriff auf ein Datenfeld benutzen wir einen Methodenaufruf –, aber der *Gewinn* ist dramatisch. Wir können nun sehr leicht die Art der Speicherung von Ausgängen im Raum ändern, ohne uns darum sorgen zu müssen, dass in der Klasse **Spiel** dadurch etwas nicht mehr funktio-

niert. Die interne Repräsentation wurde komplett von der Schnittstelle entkoppelt. Nachdem wir den Entwurf so geändert haben, wie er von Anfang an hätte sein sollen, ist der Austausch der Datenfelder für die Ausgänge durch eine **HashMap** leicht. Der geänderte Quelltext ist in Listing 8.5 zu sehen.

Listing 8.5
Quelltext der
Klasse **Raum**.

```
import java.util.HashMap;

// Klassenkommentar hier ausgelassen

public class Raum
{
    private String beschreibung;
    private HashMap<String, Raum> ausgaenge; // die Ausgänge dieses Raums

    /**
     * Erzeuge einen Raum mit einer Beschreibung. Ein Raum
     * hat anfangs keine Ausgänge. Eine Beschreibung hat die Form
     * "in einer Küche" oder "auf einem Sportplatz".
     * @param beschreibung die Beschreibung des Raums
     */
    public Raum(String beschreibung)
    {
        this.beschreibung = beschreibung;
        ausgaenge = new HashMap<>();
    }

    /**
     * Definiere einen Ausgang für diesen Raum.
     * @param richtung die Richtung, in der der Ausgang liegen soll
     * @param nachbar der Raum, der über diesen Ausgang erreicht wird
     */
    public void setzeAusgaenge(Raum norden, Raum osten, Raum sueden,
        Raum westen)
    {
        if(norden != null) {
            ausgaenge.put("north", norden);
        }
        if(osten != null) {
            ausgaenge.put("east", osten);
        }
        if(sueden != null) {
            ausgaenge.put("south", sueden);
        }
        if(westen != null) {
            ausgaenge.put("west", westen);
        }
    }

    /**
     * Liefere den Raum, den wir erreichen, wenn wir aus diesem Raum
     * in die angegebene Richtung gehen. Liefere 'null', wenn in
     * dieser Richtung kein Ausgang ist.
     * @param richtung die Richtung, in die gegangen werden soll
     * @return den Raum in der angegebenen Richtung
     */
    public Raum gibAusgang(String richtung)
    {
        return ausgaenge.get(richtung);
    }

    /**
     * @return die Beschreibung dieses Raums
     * (die dem Konstruktor übergeben wurde)
     */
    public String gibBeschreibung()
    {
        return beschreibung;
    }
}
```

Beachten Sie, dass wir diese Änderung nun vornehmen können, ohne prüfen zu müssen, ob an einer anderen Stelle etwas schiefgehen kann. Da wir nur private Aspekte der Klasse **Raum** verändert haben, die per Definition nicht von anderen

Klassen benutzt werden können, kann diese Änderung keine andere Klasse betreffen. Die Schnittstelle bleibt unverändert.

Ein Nebenprodukt unserer Änderung ist, dass die Klasse **Raum** nun sogar kürzer geworden ist. Anstelle von vier einzelnen Datenfeldern haben wir nur noch eines. Zusätzlich wurde die Methode **gibAusgang** erheblich vereinfacht.

Rufen Sie sich noch einmal in Erinnerung, dass wir all diese Änderungen nur deshalb begonnen haben, weil unsere ursprüngliche Absicht war, zusätzliche Ausgänge *nach oben* und *nach unten* zu definieren. Dies ist jetzt erheblich einfacher geworden. Da wir die Ausgänge nun in einer **HashMap** speichern, können wir die beiden zusätzlichen Ausgänge ohne jede weitere Änderung hinzufügen. Wir können auch problemlos die Information über einen Ausgang mit der Methode **gibAusgang** abfragen.

Die einzige Stelle im Quelltext, an der nach wie vor Wissen über die vier Ausgänge (*north*, *east*, *south*, *west*) „fest verdrahtet“ ist, ist in der Methode **setzeAusgaenge**. Dies ist der letzte Teil, der noch verbessert werden muss. Momentan sieht der Kopf folgendermaßen aus:

```
public void setzeAusgaenge(Raum norden, Raum osten, Raum sueden, Raum westen)
```

Diese Methode ist Teil der Schnittstelle der Klasse **Raum**. Eine Änderung an ihrer Signatur führt deshalb unweigerlich dazu, dass aufgrund der Kopplung auch andere Klassen von Änderungen betroffen sind. Wir können die Klassen einer Anwendung niemals vollständig voneinander entkoppeln, denn dann würden sie gar nicht mehr miteinander interagieren. Stattdessen versuchen wir den Grad der Kopplung so niedrig wie möglich zu halten. Wenn wir sowieso eine Änderung an der Methode **setzeAusgaenge** vornehmen wollen, um weitere Richtungen zu ermöglichen, dann ziehen wir vor, die Methode vollständig durch die folgende zu ersetzen:

```
/**
 * Definiere einen Ausgang aus diesem Raum.
 * @param richtung die Richtung, in der der Ausgang liegen soll.
 * @param nachbar der Raum, der über diesen Ausgang erreicht wird.
 */

public void setzeAusgang(String richtung, Raum nachbar)
{
    ausgaenge.put(richtung, nachbar);
}
```

So können die Ausgänge eines Raums einer nach dem anderen definiert werden und es kann eine beliebige Richtungsbenennung für einen Ausgang gewählt werden. In der Klasse **Spiel** wirken sich die Änderungen an der Schnittstelle von **Raum** folgendermaßen aus: Statt

```
labor.setzeAusgaenge(draussen, buero, null, null);
```

schreiben wir nun

```
labor.setzeAusgang("north", draussen);
labor.setzeAusgang("east", buero);
```

Insgesamt haben wir die Beschränkung, dass ein Raum nur vier Ausgänge haben kann, komplett aus dem Quelltext entfernt. Die Klasse **Raum** ist nun bereit, auch Ausgänge wie *up* und *down* oder beliebige andere aufzunehmen (*northwest*, *southeast* etc.).

Übung 8.8 Implementieren Sie die in diesem Abschnitt beschriebenen Änderungen in Ihrem eigenen *Zuul*-Projekt.

8.7 Entwurf nach Zuständigkeiten

Im vorigen Abschnitt haben wir gesehen, dass wir mit sauberer Kapselung die Kopplung und damit den Aufwand bei Änderungen in einer Anwendung erheblich reduzieren können. Kapselung ist jedoch nicht der einzige Faktor, der den Grad der Kopplung beeinflusst. Ein anderer Aspekt wird unter dem Begriff *Entwurf nach Zuständigkeiten* (*responsibility-driven design*) gefasst.

Entwurf nach Zuständigkeiten basiert auf der Idee, dass jede Klasse für den Umgang mit ihren Daten zuständig sein sollte. Wenn wir einer Anwendung neue Funktionalität hinzufügen, fragen wir uns oft, welcher Klasse wir die Methode mit der neuen Funktion hinzufügen sollten. Welche Klasse ist verantwortlich für diese Aufgabe? Die Antwort lautet: Wenn eine Klasse verantwortlich für bestimmte Daten ist, dann ist sie auch verantwortlich für den Umgang mit diesen Daten.

Wenn wir stark an Zuständigkeiten orientiert entwerfen, dann beeinflusst dies den Grad der Kopplung und damit auch den Aufwand für Änderungen oder Erweiterungen an einer Anwendung. Wie üblich werden wir dies ausführlicher an einem Beispiel demonstrieren.

8.7.1 Zuständigkeiten und Kopplung

Die Änderungen der Klasse **Raum**, die wir in Abschnitt 8.6.1 diskutiert haben, machen es recht leicht, neue Richtungen für Bewegungen nach oben oder unten zu definieren. Wir untersuchen dies anhand eines Beispiels: Nehmen Sie an, wir wollen einen neuen Raum (einen Keller) unter dem Büro definieren. Wir müssen dazu lediglich einige kleine Änderungen in der Methode **raeumeAnlegen** in der Klasse **Spiel** vornehmen, um den neuen Raum zu erzeugen und zwei Ausgänge zu setzen. Etwa so:

```
private void raeumeAnlegen()
{
    Raum draussen, hoersaal, cafeteria, labor, buero, keller;
    ...
    keller = new Raum("im Keller");
    ...
    buero.setzeAusgang("down", keller);
    keller.setzeAusgang("up", buero);
}
```

Mit der neuen Schnittstelle von **Raum** klappt das ohne Probleme. Diese Änderung ist nun sehr einfach und zeigt uns, dass der Entwurf besser wird.

Konzept

Entwurf nach Zuständigkeiten ist ein Entwurfsprozess, bei dem jeder Klasse eine klare Verantwortung zugewiesen wird. Dieser Prozess kann benutzt werden, um festzulegen, welche Klasse für welche Funktionen in einer Anwendung zuständig sein soll.

Wir können diesen Eindruck noch verstärken, indem wir die ursprüngliche Version der Methode `rauminfoAusgeben` aus Listing 8.2 mit der Methode `gibAusgaengeAlsString` vergleichen, die eine Lösung für **Übung 8.7** ist.

```
/**
 * Liefere eine Beschreibung der Ausgänge dieses Raums,
 * beispielsweise "Ausgänge: north west".
 * @return eine Beschreibung der Ausgänge dieses Raums
 */
public String gibAusgaengeAlsString()
{
    String ergebnis = "Ausgänge: ";
    if(nordausgang != null) {
        ergebnis += "north ";
    }
    if(ostausgang != null) {
        ergebnis += "east ";
    }
    if(suedausgang != null) {
        ergebnis += "south ";
    }
    if(westausgang != null) {
        ergebnis += "west ";
    }
    return ergebnis;
}
```

Listing 8.6

Die Methode `gibAusgaengeAlsString` in der Klasse `Raum`.

Weil die Informationen über seine Ausgänge nun ausschließlich im Raum selbst abgelegt sind, ist dieser auch für Informationen über Ausgänge zuständig. Der Raum kann diese Aufgabe sehr viel besser erfüllen als jedes andere Objekt, da er alle Details der internen Speicherung der Ausgänge kennt. Innerhalb der Klasse `Raum` können wir Gebrauch von dem Wissen machen, dass die Ausgänge in einer `HashMap` abgelegt sind, indem wir für die Ausgabe der Ausgänge über die Map iterieren.

Konsequenterweise ersetzen wir deshalb die Version von `gibAusgaengeAlsString` in Listing 8.6 mit der Version in Listing 8.7. Diese Methode findet alle Namen für Ausgänge in der `HashMap` (die Schlüssel in der `HashMap` sind die Namen der Ausgänge) und verkettet sie zu einer Zeichenkette, die zurückgeliefert wird. (Wir müssen die Klasse `Set` von `java.util` importieren, damit dies funktioniert.)

```
/**
 * Liefere eine Beschreibung der Ausgänge dieses Raums,
 * beispielsweise "Ausgänge: north west".
 * @return eine Beschreibung der Ausgänge dieses Raums.
 */
public String gibAusgaengeAlsString()
{
    String ergebnis = "Ausgänge: ";
    Set<String> keys = ausgaenge.keySet();
    for(String ausgang : keys){
        ergebnis += " " + ausgang;
    }
    return ergebnis;
}
```

Listing 8.7

Eine überarbeitete Fassung von `gibAusgaengeAlsString`.

Übung 8.9 Schlagen Sie die Methode `keySet` in der Dokumentation von `HashMap` nach. Was tut sie?

Übung 8.10 Erläutern Sie ausführlich und schriftlich, wie die Methode `gibAusgaengeAlsString` in Listing 8.7 arbeitet.

Wir haben nach wie vor das Ziel, die Kopplung in unserem System zu reduzieren. Dies erfordert unter anderem, dass Änderungen an der Klasse **Raum** die Klasse **Spiel** möglichst wenig beeinflussen sollten. Das ist noch verbesserungswürdig.

Momentan ist in der Klasse **Spiel** immer noch fest verdrahtet, dass die Informationen über einen Raum aus einer Raumbeschreibung und einer Beschreibung der Ausgänge bestehen:

```
System.out.println("Sie sind " + aktuellerRaum.gibBeschreibung());
System.out.println(aktuellerRaum.gibAusgaengeAlsString());
```

Was passiert, wenn wir unseren Räumen Gegenstände hinzufügen? Oder Monster? Oder andere Spieler?

Wenn wir beschreiben, was im Raum zu sehen ist, dann sollte die Liste von Gegenständen, Monstern und anderen Spielern Teil der Beschreibung des Raums sein. Wir müssten aber nicht nur Änderungen an der Klasse **Raum** vornehmen, um dies hinzuzufügen, sondern auch an dem obigen Quelltextabschnitt, in dem diese Informationen ausgegeben werden.

Auch dies ist eine Verletzung des Prinzips des Entwurfs nach Zuständigkeiten. Da die Klasse **Raum** die Informationen über einen Raum hält, sollte sie auch für die Ausgabe dieser Informationen zuständig sein. Wir können dies verbessern, indem wir der Klasse **Raum** die folgende Methode hinzufügen:

```
/**
 * Liefere eine lange Beschreibung dieses Raums, in der Form:
 * Sie sind in der Küche.
 * Ausgänge: north west
 * @return eine lange Beschreibung dieses Raumes.
 */
public String gibLangeBeschreibung()
{
    return "Sie sind " + beschreibung + ".\n" + gibAusgaengeAlsString();
}
```

In der Klasse **Spiel** schreiben wir dann

```
System.out.println(aktuellerRaum.gibLangeBeschreibung());
```

Diese „lange Beschreibung“ eines Raums schließt nun die Beschreibung des Raums selbst und die der Ausgänge ein und kann zukünftig um weitere Informationen ergänzt werden, die sich durch Erweiterungen ergeben. Wenn wir diese Erweiterungen vornehmen, brauchen wir nur noch die Klasse **Raum** zu ändern.

Übung 8.11 Implementieren Sie die in diesem Abschnitt beschriebenen Änderungen in Ihrem eigenen *Zuul*-Projekt.

Übung 8.12 Zeichnen Sie ein Objektdiagramm, das alle Objekte Ihres Spiels zu dem Zeitpunkt zeigt, zu dem das Spiel gerade gestartet wurde.

Übung 8.13 Wie ändert sich das Objektdiagramm, wenn Sie den Befehl *go* ausführen?

8.8 Änderungen lokal halten

Ein weiterer Aspekt bei den Prinzipien von Entkopplung und Zuständigkeiten ist, dass *Änderungen lokal gehalten* werden sollten. Wir streben einen Entwurf an, bei dem spätere Änderungen leicht gemacht werden, weil sie in ihren Auswirkungen lokal beschränkt bleiben.

Idealerweise sollte nur eine Klasse für eine Änderung modifiziert werden müssen. Manchmal müssen mehrere Klassen geändert werden, aber dann streben wir an, dass es möglichst wenige Klassen sind. Zusätzlich sollten die notwendigen Änderungen an anderen Klassen offensichtlich, leicht zu finden und leicht durchzuführen sein.

Wir können diesen Anspruch weitgehend erfüllen, indem wir nach Zuständigkeiten entwerfen und nach loser Kopplung und hoher Kohäsion streben. Zusätzlich sollten wir aber auch mögliche Änderungen und Erweiterungen bereits berücksichtigen, wenn wir eine Anwendung entwerfen. Es ist wichtig, mögliche Änderungen an Teilen einer Anwendung vorherzusehen, damit diese Änderungen eventuell erleichtert werden können.

Konzept

Eines der Hauptziele eines guten Klassenentwurfs lautet, **Änderungen lokal halten**: eine Änderung an einer Klasse sollte einen möglichst geringen Einfluss auf andere Klassen haben.

8.9 Implizite Kopplung

Wir haben gesehen, dass die Verwendung von öffentlichen Datenfeldern zu einer unnötig engen Kopplung zwischen Klassen führen kann. Bei einer solch engen Kopplung kann es passieren, dass mehr als eine Klasse geändert werden muss, obwohl die Änderung selbst eigentlich einfach sein sollte. Öffentliche Datenfelder sollten deshalb vermieden werden. Es gibt allerdings eine weitere Form der Kopplung, die noch unangenehmer ist: die *implizite Kopplung*.

Implizite Kopplung tritt dann auf, wenn sich eine Klasse auf interne Informationen einer anderen Klasse abstützt, diese Abhängigkeit aber nicht offensichtlich erkennbar ist. Die enge Kopplung durch die öffentlichen Datenfelder war nicht gut, aber zumindest war sie offensichtlich: Wenn wir die öffentlichen Datenfelder in einer Klasse ändern und die andere Klasse vergessen, dann lässt sich die Anwendung nicht mehr übersetzen und der Compiler meldet den Fehler. Bei impliziter Kopplung hingegen kann eine notwendige Änderung, die übersehen wurde, unentdeckt bleiben.

Wir können dieses Problem erkennen, wenn wir versuchen, dem Spiel weitere Befehle hinzuzufügen.

Nehmen Sie an, wir wollen den Befehl *look* in den Satz der gültigen Befehle aufnehmen. Die Funktion von *look* soll lediglich sein, dass die Beschreibung des Raums und seiner Ausgänge erneut ausgegeben wird (wir sehen uns – englisch *to look* – also im Raum um). Das kann dann nützlich sein, wenn wir in einem Raum schon einige Befehle eingegeben haben und die Beschreibung bereits aus dem sichtbaren Bereich des Fensters herausgeschoben wurde, wir aber nicht mehr wissen, welche Ausgänge dieser Raum hat.

Wir können ein neues Befehlswort einführen, indem wir es einfach in das Array **gueltigeBefehle** der bekannten Befehlswörter in der Klasse **Befehlswoerter** einfügen:

```
// ein konstantes Array mit den gültigen Befehlswörtern
private static final String gueltigeBefehle[] = {
    "go", "quit", "help", "look"
};
```

Dies ist ein gutes Beispiel für hohe Kohäsion: Statt die Befehlswörter in der Klasse **Parser** zu definieren, einer durchaus passenden Stelle, hat der Autor eine separate Klasse angelegt, die ausschließlich die Befehlswörter definiert. Dies macht es uns jetzt sehr leicht, die Stelle zu finden, in der die Befehlswörter definiert sind, und den neuen Befehl einzufügen. Der Autor hat offensichtlich vorausgedacht und angenommen, dass noch weitere Befehle hinzukommen werden, und entsprechend eine Struktur definiert, die dies erleichtert.

Wir können diese Änderung sofort testen. Tatsächlich passiert nichts, wenn wir das Spiel starten und **look** eingeben. Dies entspricht nicht dem Verhalten bei unbekannten Befehlen: Wenn wir einen unbekannten Befehl eingeben, sehen wir als Antwort

Ich weiß nicht, was Sie meinen ...

Somit wird der Befehl anscheinend erkannt, aber es passiert nichts, weil wir noch keine Aktion für diesen Befehl implementiert haben.

Wir können dies nun tun, indem wir für den Befehl **look** in die Klasse **Spiel** eine neue Methode einfügen:

```
private void umsehen()
{
    System.out.println(aktuellerRaum.gibLangeBeschreibung());
}
```

Sie sollten natürlich einen Kommentar für diese Methode hinzufügen. Anschließend müssen wir nur noch eine Fallunterscheidung in der Methode **verarbeiteBefehl** einfügen, die **umsehen** aufruft, wenn der Befehl **look** erkannt wurde:

```
if(befehlswort.equals("help")) {
    hilfstextAusgeben();
}
else if(befehlswort.equals("go")) {
    wechsleRaum(befehl);
}
else if(befehlswort.equals("look")) {
    umsehen();
}
else if(befehlswort.equals("quit")) {
    moechteBeenden = beenden(befehl);
}
```

Versuchen Sie es – es wird funktionieren.

Übung 8.14 Fügen Sie Ihrem eigenen *Zuul*-Projekt den Befehl *look* hinzu.

Übung 8.15 Fügen Sie dem Spiel einen weiteren Befehl hinzu. Als einfachen Anfang könnten Sie einen Befehl wie *eat* wählen, der, wenn er aufgerufen wird, zu der Meldung „Sie haben nun gegessen und sind nicht mehr hungrig.“ führt. Später können wir dies verbessern, indem der Spieler tatsächlich im Laufe der Zeit immer hungriger wird und etwas zu essen finden muss.

Die Kopplung zwischen den Klassen **Spiel**, **Parser** und **Befehlswoerter** schien bisher sehr gut zu sein – die Änderung fiel leicht und wir haben sie schnell zum Laufen gebracht. Das bereits erwähnte Problem der impliziten Kopplung wird erst offensichtlich, wenn wir nun den Befehl *help* eingeben. Die Ausgabe lautet dann:

```
Sie haben sich verlaufen. Sie sind allein. Sie irren
auf dem Unigelände herum.
Ihnen stehen folgende Befehle zur Verfügung:
go quit help
```

Hier erkennen wir ein kleines Problem. Der Hilfstext ist unvollständig: Der neue Befehl *look* ist nicht aufgeführt.

Das können wir anscheinend leicht beheben: Wir verändern einfach den Hilfstext in der Methode **hilfstextAusgeben** in **Spiel**: Das ist schnell erledigt und scheint kein großes Problem zu sein. Was aber, wenn wir den Fehler nicht bemerkt hätten? Haben Sie an dieses Problem gedacht, bevor Sie es gerade gelesen haben?

Dies ist ein grundsätzliches Problem. Jedes Mal, wenn ein neuer Befehl eingeführt wird, muss der Hilfstext angepasst werden, und das kann leicht vergessen werden. Das Programm lässt sich übersetzen und alles scheint in Ordnung zu sein. Ein Wartungsprogrammierer könnte durchaus glauben, dass seine Aufgabe erledigt ist, und das Programm freigeben, das dann einen Fehler enthält.

Dies ist ein Beispiel für implizite Kopplung: Wenn sich die Befehle ändern, dann muss der Hilfstext angepasst werden (Kopplung), aber im Quelltext gibt es keinen Hinweis darauf, dass diese Abhängigkeit besteht (deshalb implizit).

Bei einer gut entworfenen Klasse wird diese Form von Kopplung vermieden, indem den Regeln des Entwurfs nach Zuständigkeiten gefolgt wird: Da die Klasse **Befehlswoerter** verantwortlich ist für die Befehlswörter, sollte sie auch für die Darstellung der Befehle zuständig sein. Deshalb fügen wir der Klasse **Befehlswoerter** die folgende Methode hinzu:

```
/*
 * Gib alle gültigen Befehlswörter auf der Konsole aus.
 */
public void alleAusgeben()
{
    for(String befehl : gueltigeBefehle) {
        System.out.print(befehl + " ");
    }
    System.out.println();
}
```

Die Idee ist dann, dass die Methode `hilfstextAusgeben` in der Klasse `Spiel` nicht einen fest definierten Text mit den Befehlswörtern ausgibt, sondern eine Methode an der Klasse `Befehlswoerter` aufruft, die alle Befehlswörter ausgibt. Dies bewirkt, dass immer die korrekten Befehlswörter ausgegeben werden und der Hilfstext auch nach Einführung eines neuen Befehls korrekt ist.

Als letztes Problem bleibt nur noch, dass das `Spiel`-Objekt keine Referenz auf das `Befehlswoerter`-Objekt hat. Sie sehen im Klassendiagramm (Abbildung 8.1), dass es keinen Pfeil von `Spiel` zu `Befehlswoerter` gibt. Dies zeigt uns, dass die Klasse `Spiel` nichts von der Existenz der Klasse `Befehlswoerter` weiß. Stattdessen benutzt sie einen Parser, und der hat Befehlswörter.

Wir könnten nun dem Parser eine Methode hinzufügen, mit der ein `Spiel`-Objekt das `Befehlswoerter`-Objekt vom Parser erfragen kann. Dies würde aber den Grad der Kopplung in unserem System erhöhen: `Spiel` würde dann von `Befehlswoerter` abhängen, was vorher nicht der Fall war. Wir würden den Effekt im Klassendiagramm sehen können: Es würde zusätzlich ein Pfeil von `Spiel` zu `Befehlswoerter` auftauchen.

Die Pfeile im Klassendiagramm sind tatsächlich ein guter erster Hinweis darauf, wie eng gekoppelt ein Programm ist. Je mehr Pfeile, desto enger die Kopplung. Als Richtlinie für guten Klassenentwurf sollten wir deshalb Diagramme mit möglichst wenigen Pfeilen anstreben.

Also ist es gut, dass `Spiel` keine Referenz auf `Befehlswoerter` hat! Wir sollten das nicht ändern. Aus Sicht von `Spiel` ist die Tatsache, dass der Parser eine Klasse `Befehlswoerter` benutzt, ein Implementierungsdetail des Parsers. Der Parser liefert Befehle, und ob er dazu die Hilfe von weiteren Klassen benötigt, ist vollständig seiner Implementierung überlassen.

In einem guten Entwurf kommuniziert `Spiel` mit dem `Parser` und dieser dann wiederum mit `Befehlswoerter`. Wir können dies umsetzen, indem wir die folgenden Zeilen in der Methode `hilfstextAusgeben` in `Spiel` einfügen:

```
System.out.println("Ihnen stehen folgende Befehle zur Verfügung:");
parser.zeigeBefehle();
```

Es fehlt dann nur die Methode `zeigeBefehle` in der Klasse `Parser`, die diese Anfrage an die Klasse `Befehlswoerter` weiterleitet. Hier ist die vollständige Methode für die Klasse `Parser`:

```
/**
 * Gib eine Liste der bekannten Befehlswörter aus.
 */
public void zeigeBefehle()
{
    befehle.alleAusgeben();
}
```

Übung 8.16 Implementieren Sie die verbesserte Version der Ausgabe der Befehle, wie wir sie in diesem Abschnitt beschrieben haben.

Übung 8.17 Wenn Sie nun einen weiteren Befehl hinzufügen: Müssen Sie immer noch die Klasse `Spiel` ändern? Warum?

Die vollständige Implementierung aller in diesem Kapitel diskutierten Änderungen finden Sie im Begleitmaterial im Projekt *Zuul-besser*. Wenn Sie alle Übungen durchgeführt haben, können Sie das Projekt ignorieren und weiter mit Ihrem eigenen arbeiten. Wenn Sie die Übungen nicht gemacht haben, aber die folgenden Übungen als Programmieraufgabe durchführen wollen, dann können Sie das Projekt *Zuul-besser* als Startpunkt nehmen.

8.10 Vorausdenken

Der Entwurf, den wir nun erstellt haben, stellt eine erhebliche Verbesserung gegenüber der ursprünglichen Version dar. Er kann aber immer noch verbessert werden.

Eine Eigenschaft eines guten Softwareentwicklers ist die Fähigkeit zum Vorausdenken. Was könnte sich ändern? Welche Teile werden voraussichtlich während der Lebenszeit des Programms unverändert bleiben?

Eine Annahme, die wir in den meisten unserer Klassen hart verdrahtet haben, ist, dass wir dieses Spiel textbasiert mit der Ein- und Ausgabe über die Konsole spielen werden. Aber wird das immer so bleiben?

Es könnte eine interessante Erweiterung sein, später eine grafische Benutzeroberfläche mit Menüs, Knöpfen und Bildern hinzuzufügen. In dem Fall würden wir die Ausgabe nicht mehr über das Terminal vornehmen wollen. Wir könnten immer noch Befehlswörter haben und sie immer noch ausgeben wollen, wenn ein Spieler den Befehl *help* eingibt. Aber dann eher in einem Textfeld innerhalb eines Fensters statt einer Ausgabe über `System.out.println`.

In einem guten Entwurf sollten alle Informationen über die Benutzungsschnittstelle in einer Klasse oder zumindest in einer klar definierten Menge von Klassen gekapselt sein. Unsere Lösung in Abschnitt 8.9 beispielsweise, die Methode `alleAusgeben` in der Klasse `Befehlswoerter`, folgt dieser Entwurfsregel nicht. Es wäre schön zu definieren, dass `Befehlswoerter` zuständig ist für das *Produzieren* (aber nicht das *Ausgeben*!) der Befehlsliste, aber die Klasse `Spiel` sollte darüber entscheiden, wie diese Information dem Spieler gegenüber dargestellt wird.

Wir können dies leicht ändern, indem wir die Methode `alleAusgeben` so ändern, dass sie die Befehlswörter als Zeichenkette zurückliefert, statt sie selbst auszugeben. (Wir sollten sie dann besser in `gibBefehlsliste` umbenennen.) Diese Zeichenkette kann dann in der Methode `hilfstextAusgeben` in `Spiel` ausgegeben werden.

Beachten Sie, dass uns diese Änderung momentan keinen Gewinn liefert, wir aber möglicherweise in der Zukunft von diesem besseren Entwurf profitieren könnten.

Übung 8.18 Implementieren Sie die vorgeschlagene Änderung. Stellen Sie sicher, dass das Programm wie vorher abläuft.

Übung 8.19 Finden Sie etwas über das Entwurfsmuster *Model-View-Controller* heraus. Sie können eine Websuche vornehmen, um an Informationen zu kommen, oder eine beliebige andere Quelle benutzen. Wie steht dieses Muster mit der hier geführten Diskussion in Beziehung? Was schlägt das Muster vor? Wie könnte es auf dieses Projekt angewendet werden? (*Diskutieren* Sie seine Anwendung in diesem Projekt lediglich, da eine tatsächliche Implementierung eine anspruchsvolle Zusatzaufgabe wäre.)

Konzept

Kohäsion von Methoden: Eine Methode mit hoher Kohäsion ist verantwortlich für genau eine wohldefinierte Aufgabe.

8.11 Kohäsion

Wir haben den Grundgedanken von Kohäsion in Abschnitt 8.3 vorgestellt: Eine Programmeinheit sollte immer nur für genau eine Aufgabe zuständig sein. Wir werden dieses Prinzip nun näher beleuchten und uns einige Beispiele ansehen.

Das Prinzip der Kohäsion kann auf Klassen und auf Methoden angewendet werden: Sowohl Klassen als auch Methoden sollten einen hohen Grad an Kohäsion aufweisen.

8.11.1 Kohäsion von Methoden

Wenn wir über die Kohäsion von Methoden sprechen, dann wollen wir das Ideal formulieren, dass eine Methode nur für genau eine wohldefinierte Aufgabe zuständig ist.

Wir können einige Beispiele für Methoden mit hoher Kohäsion in der Klasse **Spiel** sehen. Diese Klasse hat eine private Methode **willkommenstextAusgeben** für den Eröffnungstext, die aufgerufen wird, wenn ein Spiel in der Methode **spielen** gestartet wird (Listing 8.8).

Aus funktionaler Sicht hätten wir die Anweisungen aus der Methode **willkommenstextAusgeben** auch direkt in der Methode **spielen** angeben können; das hätte das gleiche Ergebnis geliefert und wäre mit einem Methodenaufruf weniger ausgekommen. Das Gleiche kann übrigens auch für die Methode **verarbeiteBefehl** gesagt werden, die ebenfalls aus der Methode **spielen** heraus aufgerufen wird: Auch diese Anweisungen hätten direkt in der Methode **spielen** angegeben werden können.

Ein Quelltextabschnitt ist jedoch sehr viel leichter zu verstehen und Änderungen sind leichter vorzunehmen, wenn kurze Methoden mit hoher Kohäsion verwendet wurden. In der gewählten Struktur der Methoden sind die einzelnen Methoden relativ kurz und auch gut zu verstehen, ihre Namen geben ihre Funktion deutlich wieder. Diese Eigenschaften sind für Wartungsprogrammierer eine große Hilfe.


```

/**
 * Die Hauptmethode zum Spielen. Läuft bis zum Ende des Spiels
 * in einer Schleife.
 */
public void spielen()
{
    willkommenstextAusgeben();

    // Die Hauptschleife. Hier lesen wir wiederholt Befehle ein
    // und führen sie aus, bis das Spiel beendet wird.

    boolean beendet = false;
    while (!beendet) {
        Befehl befehl = parser.liefereBefehl();
        beendet = verarbeiteBefehl(befehl);
    }
    System.out.println("Danke für dieses Spiel. Auf Wiedersehen.");
}

```

```

/**
 * Einen Begrüßungstext für den Spieler ausgeben.
 */
private void willkommenstextAusgeben()
{
    System.out.println();
    System.out.println("Willkommen zu Zuul!");
    System.out.println("Zuul ist ein neues, unglaublich langweiliges Spiel.");
    System.out.println("Tippen Sie 'help', wenn Sie Hilfe brauchen.");
    System.out.println();
    System.out.println(aktuellerRaum.gibLangeBeschreibung());
}

```

Listing 8.8

Zwei Methoden mit einem hohen Grad an Kohäsion.

8.11.2 Kohäsion von Klassen

Die Regel für die Kohäsion von Klassen besagt, dass jede Klasse genau eine wohldefinierte Einheit aus dem Anwendungsbereich modellieren sollte.

Als ein Beispiel für die Kohäsion von Klassen werden wir nun eine weitere Erweiterung am *Zuul*-Projekt diskutieren. Wir wollen *Gegenstände* einführen. Jeder Raum kann einen Gegenstand enthalten und jeder Gegenstand hat eine Beschreibung und ein Gewicht. Das Gewicht eines Gegenstands kann später darüber entscheiden, ob er aufgehoben werden kann oder nicht.

Ein naiver Ansatz wäre, zwei neue Datenfelder in der Klasse **Raum** einzuführen: **gegenstandBeschreibung** und **gegenstandGewicht**. Wir könnten dann die Details der Gegenstände in den Räumen festlegen und sie ausgeben lassen, sobald ein Raum betreten wird.

Dieser Ansatz würde aber nicht zu einem hohen Grad an Kohäsion führen: Die Klasse **Raum** würde dann sowohl einen Raum als auch einen Gegenstand beschreiben. Das würde auch bedeuten, dass ein Gegenstand an einen bestimmten Raum gebunden ist, und das soll vielleicht nicht der Fall sein.

Ein besserer Entwurf würde eine neue Klasse für Gegenstände einführen, die man **Gegenstand** nennen könnte. Diese Klasse würde Datenfelder für die Beschreibung und das Gewicht definieren und ein Raum würde einfach eine Referenz auf einen solchen Gegenstand enthalten.

Konzept

Kohäsion von Klassen: Eine Klasse mit hoher Kohäsion repräsentiert genau eine wohldefinierte Einheit.

Übung 8.20 Erweitern Sie entweder Ihr eigenes Projekt oder das vorgegebene Projekt *Zuul-besser* so, dass jeder Raum einen Gegenstand enthält. Gegenstände haben eine Beschreibung und ein Gewicht. Bei der Erzeugung der Räume und dem Setzen der Ausgänge sollten auch Gegenstände für das Spiel erzeugt werden. Wenn ein Spieler einen Raum betritt, sollten auch die Informationen über den Gegenstand in diesem Raum angezeigt werden.

Übung 8.21 Wie sollten die textbasierten Informationen über einen Gegenstand in einem Raum erzeugt werden? Welche Klasse sollte die Zeichenkette erzeugen, die den Gegenstand beschreibt? Welche Klasse sollte diese ausgeben? Warum? Erklären Sie das schriftlich. Wenn Sie beim Beantworten dieser Fragen das Gefühl bekommen, dass Sie Ihre Implementierung anpassen sollten, dann tun Sie das ruhig.

Der eigentliche Vorteil der Trennung von Raum und Gegenstand in unserem Entwurf wird erst deutlich, wenn wir die Anforderungen an die Erweiterung etwas abändern. In einer weiteren Version des Spiels wollen wir nicht nur einen einzelnen Gegenstand in einem Raum zulassen, sondern eine beliebige Anzahl. In einem Entwurf mit einer eigenen Klasse für Gegenstände ist dies einfach: Wir erzeugen mehrere **Gegenstand**-Objekte und legen sie in einer Sammlung von Gegenständen im Raum ab.

Mit dem ersten, naiven Ansatz wäre es fast unmöglich, diese Änderung durchzuführen.

Übung 8.22 Modifizieren Sie das Projekt so, dass ein Raum beliebig viele Gegenstände enthalten kann. Benutzen Sie dafür eine Sammlung. Stellen Sie sicher, dass ein Raum eine Methode **gegenstandAblegen** hat, mit der ein Gegenstand in einem Raum abgelegt werden kann. Sorgen Sie auch dafür, dass alle Gegenstände angezeigt werden, wenn ein Spieler einen Raum betritt.

8.11.3 Kohäsion für bessere Lesbarkeit

Hohe Kohäsion verbessert einen Entwurf in vielerlei Hinsicht. Die beiden wichtigsten Aspekte sind *Lesbarkeit* und *Wiederverwendung*.

Die Methode **willkommenstextAusgeben**, die in Abschnitt 8.11.1 erwähnt wurde, ist ein deutliches Beispiel dafür, dass die Erhöhung der Kohäsion den Quelltext lesbarer und damit einfacher zu verstehen und zu warten macht.

Das Beispiel für Kohäsion von Klassen in Abschnitt 8.11.2 enthält auch einen Lesbarkeitsaspekt. Wenn eine eigene Klasse **Gegenstand** gegeben ist, dann kann ein Wartungsprogrammierer leicht die Stelle finden, an der die Eigenschaften von Gegenständen geändert werden können. Die Kohäsion von Klassen erhöht ebenfalls die Lesbarkeit eines Programms.

8.11.4 Kohäsion für Wiederverwendbarkeit

Der zweite große Vorteil von Kohäsion ist ein hohes Potenzial für Wiederverwendung.

Das Beispiel für Kohäsion von Klassen in Abschnitt 8.11.2 dient hier als Beispiel: Indem wir eine eigene Klasse **Gegenstand** definieren, können wir mehrere Gegenstände erzeugen und somit den gleichen Quelltext für mehr als einen Gegenstand verwenden.

Wiederverwendung ist auch ein wichtiger Aspekt bei der Kohäsion von Methoden. Stellen Sie sich eine Methode in der Klasse **Raum** mit folgendem Kopf vor:

```
public Raum verlasseRaum(String richtung)
```

Diese Methode könnte den Raum in der angegebenen Richtung liefern (sodass er als neuer aktueller Raum verwendet werden kann) und außerdem eine Beschreibung des Raums ausgeben, den wir gerade betreten.

Das ist ein möglicher Entwurf und er kann auch funktionieren. In unserer Version haben wir diese Aufgabe auf zwei Methoden aufgeteilt:

```
public Raum gibAusgang(String richtung)
public String gibLangeBeschreibung()
```

Die erste Methode ist zuständig für die Lieferung des nächsten Raums, während die zweite eine Beschreibung des Raums liefert.

Der Vorteil an diesem Entwurf ist, dass die getrennten Vorgänge besser wiederverwendet werden können. Die Methode **gibLangeBeschreibung** beispielsweise wird nicht nur in der Methode **wechsleRaum** benutzt, sondern auch in der Methode **willkommenstextAusgeben** und in der Implementierung für den Befehl *look*. Das ist nur deshalb möglich, weil sie einen hohen Grad an Kohäsion aufweist. Ihre Wiederverwendung wäre nicht möglich in der Version mit der Methode **verlasseRaum**.

Übung 8.23 Implementieren Sie den Befehl *back*. Dieser Befehl hat kein zweites Wort. Die Eingabe des Befehls *back* soll den Spieler in den Raum zurückbringen, in dem er zuletzt gewesen ist.

Übung 8.24 Testen Sie Ihren neuen Befehl. Verhält er sich wie erwartet? Testen Sie auch für den Fall, dass der Befehl nicht korrekt verwendet wird. Was macht zum Beispiel Ihr Programm, wenn ein Spieler nach *back* noch ein zweites Wort angibt? Verhält es sich vernünftig?

Übung 8.25 Was macht Ihr Programm, wenn *back* zweimal eingegeben wird? Ist dieses Verhalten sinnvoll?

Übung 8.26 Zusatzaufgabe. Implementieren Sie den Befehl *back* so, dass eine wiederholte Anwendung den Spieler mehrere Räume zurückversetzt, bei genügend häufiger Anwendung sogar an den Spielanfang. Benutzen Sie dazu einen **Stack**. (Sie müssen vermutlich herausfinden, was ein Stack ist. Sehen Sie in der Dokumentation der Java-Bibliothek nach.)

Konzept

Refactoring ist eine Aktivität, bei der ein bestehender Entwurf restrukturiert wird, um die Qualität des Klassenentwurfs in Hinsicht auf vorzunehmende Änderungen und Erweiterungen zu erhalten.

8.12 Refactoring

Beim Entwurf von Anwendungen sollten wir versuchen, vorausschauend zu denken und mögliche zukünftige Änderungen vorherzusehen; wir sollten lose gekoppelte Klassen mit hoher Kohäsion entwerfen sowie Methoden, die Änderungen leicht machen. Das ist ein hehres Ziel, aber wir können nicht alle zukünftig möglichen Änderungen vorhersehen, und es ist nicht möglich, sich auf alle denkbaren Erweiterungen einzustellen.

Deshalb ist *Refactoring* wichtig.

Refactoring wird die Aktivität genannt, bei der bestehende Klassen und Methoden restrukturiert werden, um sie geänderten Umständen und Anforderungen anzupassen. Während der Lebensdauer einer Anwendung wird Funktionalität oft erst nach und nach hinzugefügt. Sehr häufig wachsen dadurch Methoden und Klassen als Nebeneffekt in ihrer Länge.

Die Versuchung für einen Wartungsprogrammierer ist groß, neue Anweisungen in bestehende Klassen und Methoden einzufügen. Nach einiger Zeit leidet aber der Grad der Kohäsion darunter. Wenn immer mehr Anweisungen zu einer Methode oder einer Klasse hinzugefügt werden, dann ist irgendwann die Wahrscheinlichkeit sehr hoch, dass sie mehr als eine klar definierte Aufgabe ausfüllt bzw. nicht mehr eine klare Einheit darstellt.

Refactoring ist das Überdenken und Restrukturieren von Klassen- und Methodenstrukturen. Sehr häufig wird dabei eine Klasse in zwei Klassen aufgespalten oder eine Methode in zwei oder mehr Methoden aufgeteilt. Refactoring schließt auch das Zusammenfassen mehrerer Klassen oder Methoden zu einer einzelnen ein, aber dieser Fall ist seltener.

8.12.1 Refactoring und Testen

Bevor wir ein Beispiel für Refactoring betrachten, sollten wir kurz über die Tatsache nachdenken, dass wir ein Programm meist dann restrukturieren, wenn wir eine größere Änderung an etwas vornehmen wollen, das bereits läuft. Wenn etwas geändert wird, dann besteht die Möglichkeit neuer Fehler. Deshalb sollten wir sehr vorsichtig vorgehen und vor der Restrukturierung einen Satz Tests für die laufende Anwendung vorliegen haben. Wenn noch keine Tests existieren, dann sollten wir erst entscheiden, wie wir auf vernünftige Art und Weise die Funktionalität des Programms testen können, und diese Tests aufzeichnen (zum Beispiel indem wir sie schriftlich festhalten), sodass wir die gleichen Tests später wiederholen können. Wir werden auf das Testen im nächsten Kapitel noch näher eingehen. Wenn Sie bereits mit automatisiertem Testen vertraut sind, sollten Sie automatisierte Tests verwenden. Andernfalls reicht manuelles (systematisches) Testen durchaus im Moment aus.

Sobald Sie sich auf einen Satz Tests geeinigt haben, sollten Sie mit der Umstrukturierung beginnen. Idealerweise folgt das Refactoring dann in zwei Schritten:

- Im ersten Schritt werden Umstrukturierungen vorgenommen, um die interne Struktur des Quelltextes zu verbessern, aber ohne Änderungen an der Funktionalität der Anwendung vorzunehmen. Anders gesagt: Das Programm sollte sich bei der Ausführung genauso verhalten wie zuvor. Nachdem dieser Schritt abgeschlossen ist, sollten die zuvor erstellten Tests wiederholt werden, um sicherzustellen, dass keine unbeabsichtigten neuen Fehler eingeführt wurden.
- Der zweite Schritt wird erst dann ausgeführt, wenn wir die Basisfunktionalität in der restrukturierten Version wiederhergestellt haben. Wir können dann beruhigt Erweiterungen am Programm vornehmen. Nachdem dies geschehen ist, sollte natürlich auch die neue Version gründlich getestet werden.

Mehrere gleichzeitige Änderungen (restrukturieren und neue Funktionalität einführen) machen es schwieriger, eventuell auftauchende Fehler zu lokalisieren.

Übung 8.27 Welche grundlegenden Tests für die Basisfunktionalität der aktuellen Version unseres Spiels sollten wir erstellen?

8.12.2 Ein Beispiel für Refactoring

Als ein Beispiel sollten wir mit der Erweiterung unseres Spiels um Gegenstände fortfahren. In Abschnitt 8.11.2 haben wir Gegenstände eingeführt und eine Struktur vorgeschlagen, in der jeder Raum beliebig viele Gegenstände enthalten kann. Ein logischer nächster Schritt wäre nun, wenn ein Spieler diese Gegenstände aufheben und mit sich herumtragen könnte. Dies könnte informell etwa so spezifiziert werden:

- Ein Spieler kann Gegenstände im aktuellen Raum aufheben.
- Ein Spieler kann beliebig viele Gegenstände bei sich tragen, jedoch nur bis zu einem Maximalgewicht.
- Einige Gegenstände können nicht aufgehoben werden.
- Der Spieler kann Gegenstände im aktuellen Raum ablegen.

Um dies zu erreichen, können wir Folgendes tun:

- Falls noch nicht geschehen, führen wir eine Klasse **Gegenstand** in unser Projekt ein. Ein Gegenstand hat, wie bereits vorher beschrieben, eine Beschreibung (als Zeichenkette) und ein Gewicht (eine ganze Zahl).
- Wir sollten für einen Gegenstand auch einen Namen einführen. Dies ermöglicht dem Spieler, über einen kürzeren Namen als über die Beschreibung auf den Gegenstand Bezug zu nehmen. Wenn beispielsweise im aktuellen Raum ein Buch liegt, dann könnten die Datenfelder des Gegenstands so aussehen:

name: **Buch**

beschreibung: **ein altes, staubiges Buch mit Ledereinband**

gewicht: **1200**

Wenn wir einen Raum betreten, können wir die Beschreibung eines Gegenstands ausgeben, damit der Spieler informiert ist. Für Befehle ist ein Name jedoch einfacher zu benutzen. Beispielsweise kann ein Spieler dann *take Buch* eintippen, um das Buch aufzuheben.

- Wir können sicherstellen, dass ein Spieler einen Gegenstand nicht aufheben kann, indem wir ihn sehr schwer machen (schwerer, als ein Spieler tragen kann). Oder sollten wir ein weiteres boolesches Datenfeld **aufhebbbar** definieren? Welches ist Ihrer Meinung nach der bessere Entwurf? Ist das wichtig? Versuchen Sie dies zu beantworten, indem Sie überlegen, welche Änderungen zukünftig noch an dem Spiel vorgenommen werden könnten.
- Wir fügen die Befehle *take* und *drop* ein, um Gegenstände aufheben bzw. wieder ablegen zu können. Beide Befehle haben als zweites Wort den Namen eines Gegenstands.
- An irgendeiner Stelle müssen wir ein Datenfeld (für eine Sammlung) einführen, um die aktuell vom Spieler getragenen Gegenstände zu halten. Wir müssen auch ein Datenfeld einführen, das die maximale Tragkraft eines Spielers festlegt, damit wir es bei jedem Versuch, etwas aufzuheben, überprüfen können. Wo sollten diese Datenfelder definiert werden? Denken Sie auch hier wieder an mögliche zukünftige Erweiterungen, um Ihre Entscheidung zu erleichtern.

Die letztgenannte Aufgabe wollen wir nun etwas ausführlicher besprechen, um die Idee des Refactoring zu illustrieren.

Wenn ein Spieler Gegenstände bei sich tragen können soll, dann lautet die erste Entwurfsfrage: An welcher Stelle sollten wir die Datenfelder einfügen, in denen das aktuelle Gepäck und die maximale Tragkraft gehalten wird? Ein schneller Blick auf die bestehenden Klassen zeigt uns, dass **Spiel** eigentlich die einzige Klasse ist, in die diese Daten passen. Diese Informationen können nicht in **Raum**, **Gegenstand** oder **Befehl** gespeichert werden, da es von diesen Klassen im Laufe eines Spiels viele Instanzen geben wird, die nicht immer zugreifbar sind. In **Parser** oder **Befehlswoerter** passen sie auch nicht.

Ein weiteres Argument für die Platzierung in **Spiel** ist, dass dort bereits der aktuelle Raum (in dem sich der Spieler gerade befindet) gehalten wird, sodass auch die aktuellen Gegenstände (die der Spieler gerade bei sich trägt) dort hinzupassen scheinen.

Dieser Ansatz ist denkbar. Er führt allerdings nicht zu einem guten Entwurf. Die Klasse **Spiel** ist schon recht umfangreich, es gibt einige Hinweise, dass sie bereits zu groß ist. Zusätzliche Erweiterungen würden das nicht verbessern.

Wir sollten uns noch einmal fragen, zu welcher Klasse oder welchem Objekt die genannten Informationen gehören. Wenn wir genauer darüber nachdenken, was hier verwaltet werden soll (aufgenommene Gegenstände, Tragkraft), dann erkennen wir, dass es Informationen über einen *Spieler* sind! Die logische Konsequenz ist deshalb (dem Entwurf nach Zuständigkeiten folgend), eine neue Klasse **Spieler** einzuführen. Wir können die genannten Daten dann in der Klasse **Spieler** definieren und zum Spielstart ein Objekt dieser Klasse erzeugen, das die Daten verwaltet.

Das bestehende Datenfeld **aktuellerRaum** hält ebenfalls eine Information über den Spieler: seine aktuelle Position. Konsequenterweise sollte dieses Datenfeld auch in die Klasse **Spieler** wandern.

Wenn wir diesen Entwurf betrachten, dann können wir sagen, dass er besser dem Prinzip von klaren Zuständigkeiten entspricht. Wer sollte verantwortlich sein für Informationen über Spieler? Natürlich die Klasse **Spieler**.

In der ursprünglichen Version hatten wir nur eine Information über einen Spieler zu speichern – den aktuellen Raum. Man kann diskutieren, ob dies nicht bereits eine Klasse **Spieler** erfordert hätte. Es gibt Argumente dafür und dagegen. Es wäre ein besserer Entwurf gewesen, also hätten wir es bereits tun können. Aber eine Klasse mit nur einem Datenfeld und keinen echten eigenen Methoden könnte man auch als zu aufwendig bezeichnen.

Manchmal gibt es solche Grauzonen, in denen für oder gegen eine Entwurfsentscheidung argumentiert werden kann. Nachdem wir nun aber die neuen Datenfelder hinzufügen wollen, ist klar, dass wir gute Argumente für eine Klasse **Spieler** haben. Diese kann die genannten Daten speichern und Methoden anbieten wie **gegenstandAblegen** und **gegenstandAufnehmen** (die eine Prüfung des Gewichts einschließen und **false** liefern könnte, wenn der Gegenstand zu schwer ist).

Wenn wir die Klasse **Spieler** einführen und das Datenfeld **aktuellerRaum** aus **Spiel** in **Spieler** verlagern, nehmen wir ein Refactoring vor. Wir haben die Daten so restrukturiert, dass der Gesamtentwurf den geänderten Anforderungen besser gerecht wird.

Weniger gut ausgebildete (oder einfach faule) Programmierer hätten das Datenfeld **aktuellerRaum** möglicherweise am alten Platz gelassen, da das Programm auch dann offensichtlich noch läuft und kein zwingender Anlass für diese Änderung besteht. Dieser Programmierer würde sich mit einem schlechten Entwurf abfinden.

Den Effekt dieser Änderung merken wir erst, wenn wir einen Schritt weiter denken. Nehmen Sie an, dass wir das Spiel nun so erweitern wollen, dass es mehrere Spieler geben kann. Mit unserem neuen Entwurf ist das plötzlich sehr einfach. Wir haben bereits eine Klasse **Spieler** (die Klasse **Spiel** hält eine Instanz dieser Klasse) und können sehr leicht mehrere **Spieler**-Objekte erzeugen, die wir dann in einer Sammlung in der Klasse **Spiel** halten können. Jedes **Spieler**-Objekt hätte seinen eigenen aktuellen Raum, eigene Gegenstände und eine eigene Tragkraft. Verschiedene Spieler könnten eine unterschiedliche Tragkraft haben, wodurch die Tür aufgestoßen wird für weitere Änderungen, bei denen unterschiedliche Spieler unterschiedliche Fähigkeiten haben – mit der Tragkraft als einer von vielen.

Der faule Programmierer hingegen, der das Datenfeld **aktuellerRaum** in der Klasse **Spiel** gelassen hat, hat nun ein ernsthaftes Problem: Da das gesamte Spiel nur einen aktuellen Raum hat, können die aktuellen Positionen der verschiedenen Spieler nicht leicht gespeichert werden. Ein schlechter Entwurf fällt oft später wieder auf uns zurück und kann dann eine Menge Mehrarbeit bedeuten.

Gutes Refactoring ist ebenso eine Frage der Einstellung wie der technischen Fähigkeiten. Während wir eine Anwendung ändern oder erweitern, sollten wir uns ständig fragen, ob der ursprüngliche Entwurf noch immer die beste Entscheidung darstellt. Wenn sich die Funktionalität ändert, ändern sich auch die Argumente für oder gegen eine Entwurfsentscheidung. Ein guter Entwurf für eine einfache Anwendung kann zu einem schlechten werden, wenn die Anwendung erweitert wird.

Das Erkennen notwendiger Änderungen und das tatsächliche Durchführen des Refactoring am Quelltext kann uns letztlich eine Menge Zeit und Aufwand ersparen. Je eher wir einen Entwurf aufräumen, desto mehr Aufwand ersparen wir uns üblicherweise.

Wir sollten ständig bereit sein, Methoden zu *extrahieren* (eine Reihe von Anweisungen aus dem Rumpf einer Methode in eine neue, unabhängige Methode auslagern) und Klassen zu extrahieren (Teile einer Klasse ausschneiden und in eine eigene Klasse auslagern). Solche Refactorings halten unsere Entwürfe sauber und sparen letztlich Arbeit. Eines ist dabei klar: Refactoring kann uns auf lange Sicht das Leben schwer machen, wenn wir es nicht schaffen, die überarbeitete Version im Vergleich zur Originalversion zu testen. Wir sollten immer, wenn wir uns auf ein größeres Refactoring einlassen, vorher sicherstellen, dass wir vor und nach der Änderung gut getestet haben. Manuelle Tests (durch interaktives Erzeugen und Testen von Objekten) erweisen sich schon sehr schnell als mühsam. Wir werden im nächsten Kapitel näher darauf eingehen, wie wir unsere Tests (durch Automatisieren) verbessern können.

Übung 8.28 Restrukturieren Sie Ihr Projekt für eine neue Klasse **Spieler**. Ein **Spieler**-Objekt sollte mindestens den aktuellen Raum des Spielers halten, aber Sie können auch weitere Informationen wie etwa den Namen des Spielers vorsehen.

Übung 8.29 Implementieren Sie eine Erweiterung, bei der ein Spieler genau einen Gegenstand aufnehmen kann. Dazu müssen Sie zwei weitere Befehle implementieren: *take* und *drop*.

Übung 8.30 Erweitern Sie Ihre Implementierung so, dass ein Spieler beliebig viele Gegenstände aufnehmen kann.

Übung 8.31 Führen Sie die Beschränkung ein, dass ein Spieler nur Gegenstände bis zu einem maximalen Gewicht an sich nehmen kann. Die Tragkraft eines Spielers sollte eines seiner Attribute sein.

Übung 8.32 Implementieren Sie einen Befehl *status*, der alle aufgenommenen Gegenstände auflistet und ihr Gesamtgewicht anzeigt.

Übung 8.33 Legen Sie in einem Raum einen *magischen Muffin* ab. Fügen Sie den Befehl *eat muffin* hinzu. Wenn ein Spieler den Muffin findet und isst, dann erhöht dies seine maximale Tragkraft. (Sie können dies so anpassen, dass es gut in Ihren Spielentwurf hineinpasst.)

8.13 Refactoring für Sprachunabhängigkeit

Ein Detail des *Zuul*-Spiels, zu dem wir bisher noch nichts gesagt haben, ist die Vermischung von Deutsch und Englisch in der Benutzungsschnittstelle. Die Kommandos werden in Englisch angegeben, die Beschreibungen der Räume sind aber in Deutsch. Möglicherweise hat es sich der Übersetzer des Originalspiels nur leicht machen wollen; oder die Kommandostruktur wurde beibehalten, weil der Befehl „go east“ einfach griffiger und korrekter ist als „gehe ost“ (statt „gehe nach Osten“ oder „gehe ostwärts“). Auf alle Fälle sind die englischen Befehle eingebettet sowohl in die Klasse **Befehlswoerter** (hier werden die erlaubten Befehle gehalten) als auch in der Klasse **Spiel**, in der explizit jeder eingegebene Befehl mit einer Menge von englischen Wörtern abgeglichen wird. Wenn wir die Schnittstelle komplett auf deutsche Kommandos (oder die einer anderen Sprache) umstellen woll-

ten, dann müssten wir alle Stellen im Quelltext finden und ändern, an denen Befehlswörter verwendet werden. Dies ist ein weiteres Beispiel für implizite Kopplung, die wir in Kapitel 8.9 diskutiert haben.

Wenn das Programm sprachunabhängig sein soll, dann sollte es idealerweise nur genau eine Stelle im Quelltext geben, an der die Befehlswörter gespeichert sind, damit an allen anderen Stellen auf sprachunabhängige Befehle Bezug genommen werden kann. Ein Programmiersprachenkonstrukt, das dies ermöglicht, sind *Aufzählungstypen* (*enumerated types* oder kurz *enums*). Wir werden es uns mithilfe der Projekte *Zuul-mit-Enums* ansehen.

8.13.1 Aufzählungstypen

Listing 8.9 zeigt die Java-Definition eines Aufzählungstyps **Befehlswort**.

```
/**
 * Repräsentationen für alle gültigen Befehlswörter des Spiels.
 *
 * @author Michael Kölling und David J. Barnes
 * @version 2016.02.29
 */
public enum Befehlswort
{
    // Ein Wert für jedes Befehlswort, plus eines für nicht
    // erkannte Befehle
    GO, QUIT, HELP, UNKNOWN;
}
```

Listing 8.9

Ein Aufzählungstyp für Befehlswörter.

In seiner simpelsten Form besteht ein Aufzählungstyp aus einer äußeren Klammer, in der das Schlüsselwort **enum** anstelle von **class** verwendet wird, und einem Rumpf, in dem einfach die Namen von Elementen aufgezählt werden, die die Werte des Typs darstellen. Nach Konvention werden diese Namen in Großbuchstaben angegeben. Wir erzeugen niemals Objekte eines Aufzählungstyps. Jeder Name innerhalb der Typdefinition repräsentiert eine eindeutige Instanz des Typs, die bereits für uns erzeugt wurde. Wir beziehen uns auf diese Instanzen mit **Befehlswort.GO**, **Befehlswort.QUIT** usw. Auch wenn die Syntax ihrer Verwendung sehr ähnlich ist, sollten wir diese Werte nicht mit den numerischen Klassenkonstanten aus Abschnitt 6.14 verwechseln. Trotz der einfachen Definition sind die Werte eines Aufzählungstyps echte Objekte und nicht das gleiche wie **int**-Werte.

Wie können wir diesen Typ **Befehlswort** nun verwenden, um die Spiellogik innerhalb von *Zuul* von einer spezifischen Sprache etwas mehr zu entkoppeln? Eine der ersten möglichen Verbesserungen können wir an den folgenden Tests vornehmen, die in der Methode **verarbeiteBefehl** der Klasse **Spiel** stehen:

```
if(befehl.istUnbekannt()) {
    System.out.println("Ich weiss nicht, was Sie meinen ...");
    return false;
}
String befehlsword = befehl.gibBefehlswort();
if (befehlsword.equals("help")) {
    hilfstextAusgeben();
}
else if (befehlsword.equals("go")) {
    wechseleRaum(befehl);
}
```

```

else if (befehlswort.equals("quit")) {
    moechteBeenden = beenden(befehl);
}

```

Wenn **befehlswort** vom Typ **Befehlswort** und nicht als **String** deklariert wird, dann können die Tests folgendermaßen umformuliert werden:

```

if(befehlswort == Befehlswort.UNKNOWN) {
    System.out.println("Ich weiss nicht, was Sie meinen ...");
    return false;
}
else if(befehlswort == Befehlswort.HELP) {
    hilfstextAusgeben();
}
else if(befehlswort == Befehlswort.GO) {
    wechsleRaum(befehl);
}
else if(befehlswort == Befehlswort.QUIT) {
    moechteBeenden = beenden(befehl);
}

```

Nachdem wir jetzt den Typ in **Befehlswort** geändert haben, könnten wir auch eine *switch-Anweisung* anstelle einer Reihe von *if*-Anweisungen verwenden. Dadurch wird die Absicht des Quelltextes ein bisschen deutlicher.²

```

switch(befehlswort) {
    case UNKNOWN:
        System.out.println("Ich weiss nicht, was Sie meinen ...");
        break;

    case HELP:
        hilfstextAusgeben();
        break;

    case GO:
        wechsleRaum(befehl);
        break;

    case QUIT:
        moechteBeenden = beenden(befehl);
        break;
}

```

Konzept

Eine **switch-Anweisung** wählt aus verschiedenen möglichen Anweisungsfolgen eine Folge zur Ausführung aus.

Die **switch**-Anweisung übernimmt nach dem Schlüsselwort **switch** in Klammern die Variable (in unserem Fall **befehlswort**) und vergleicht sie mit jedem der Werte, die nach den **case**-Schlüsselwörtern aufgeführt werden. Wenn eine Übereinstimmung festgestellt wird, wird der nachfolgende Quelltext ausgeführt. Die **break**-Anweisung bricht die **switch**-Anweisung an der aktuellen Stelle ab und die Ausführung wird nach der **switch**-Anweisung fortgesetzt. Eine genauere Beschreibung der **switch**-Anweisung finden Sie in Anhang D.

² Tatsächlich können String-Literale ebenfalls als **case**-Werte in **switch**-Anweisungen verwendet werden, um eine lange Folge von **if-else-if**-Vergleichen zu vermeiden.

Nun müssen wir uns nur noch darum kümmern, dass die vom Benutzer eingetippten Befehle auf die entsprechenden Werte von **Befehlswort** abgebildet werden. Öffnen Sie das Projekt *Zuul-mit-Enums-V1* und sehen Sie sich an, wie wir dies umgesetzt haben. Die wichtigste Änderung ist in der Klasse **Befehlswörter** zu finden. Anstatt ein Array von Strings für die Definition der gültigen Befehle zu verwenden, nehmen wir nun eine Abbildung von Strings auf **Befehlswort**-Objekte:

```
public Befehlswörter()
{
    gueltigeBefehle = new HashMap<>();
    gueltigeBefehle.put("go", Befehlswort.GO);
    gueltigeBefehle.put("help", Befehlswort.HELP);
    gueltigeBefehle.put("quit", Befehlswort.QUIT);
}
```

Der vom Benutzer eingetippte Befehl kann nun leicht in den entsprechenden Wert des Aufzählungstyps umgewandelt werden.

Übung 8.34 Untersuchen Sie im Quelltext des Projekts *Zuul-mit-Enums-V1*, wie der Typ **Befehlswort** verwendet wird. Die Klassen **Befehl**, **Befehlswörter**, **Spiel** und **Parser** wurden alle aus der Version in *Zuul-besser* angepasst, um diese Änderungen nachzuvollziehen.

Übung 8.35 Fügen Sie dem Spiel einen Befehl *look* hinzu, so wie in Abschnitt 8.9 beschrieben.

Übung 8.36 Übersetzen Sie im Spiel die verschiedenen Bezeichnungen für die Befehlswörter **GO** und **QUIT** von *go* und *quit* in deutsche Bezeichnungen. Müssen Sie ausschließlich die Klasse **Befehlswörter** ändern, um dies zu tun?

Übung 8.37 Übersetzen Sie nun auch den **Hilfe**-Befehl von *help* ins Deutsche und überprüfen Sie, ob alles noch klappt. Was bemerken Sie beim Willkommenstext, der beim Starten des Spiels ausgegeben wird?

Übung 8.38 Definieren Sie in einem neuen Projekt Ihren eigenen Aufzählungstyp **Richtung** mit den Werten **NORD**, **SUED**, **OST** und **WEST**.

8.13.2 Weitere Entkopplung der Befehlsschnittstelle

Der Aufzählungstyp **Befehlswort** hat es uns ermöglicht, die Sprache an der Benutzungsschnittstelle deutlich von der Spiellogik zu entkoppeln. Es ist fast vollständig möglich, die Befehlswörter in eine andere Sprache zu übersetzen, indem wir ausschließlich die Klasse **Befehlswörter** editieren. (An irgendeinem Punkt sollten wir auch überlegen, die Raumbeschreibungen und andere Texte aus einer Datei einzulesen, aber das lassen wir für später.) Einen weiteren Entkopplungsschritt können wir aber noch machen. Momentan müssen wir mit jedem neuen Befehlswort einen neuen Wert im Typ **Befehlswort** definieren und eine Abbildung vom Benutzerbefehl auf diesen Wert in der Klasse **Befehlswörter** definieren. Es wäre hilfreich, wenn wir den Typ **Befehlswort** selbstständiger machen würden; dazu müssten wir die Abbildung von Text auf Wert aus der Klasse **Befehlswörter** nach **Befehlswort** bewegen.

Java erlaubt bei der Definition von Aufzählungstypen sehr viel mehr als nur eine Aufzählung der Werte eines Typs. Wir werden diese Möglichkeiten nicht im Detail untersuchen, wollen Ihnen aber zumindest einen Eindruck vermitteln. Listing 8.10 zeigt eine erweiterte Version des Typs **Befehlswort**, die einer normalen Klassendefinition sehr ähnlich sieht. Sie ist im Projekt *Zuul-mit-Enums-V2* zu finden.

Listing 8.10

Verknüpfen von Benutzerbefehlen mit Werten eines Aufzählungstyps.

```
/**
 * Repräsentationen für alle gültigen Befehlswörter des Spiels,
 * zusammen mit einer Zeichenkette in einer bestimmten Sprachen.
 *
 * @author Michael Kölling und David J. Barnes
 * @version 2016.02.29
 */
public enum Befehlswort
{
    // Ein Wert für jedes Befehlswort zusammen mit der dazugehörigen
    // Benutzereingabe
    GO("go"), QUIT("quit"), HELP("help"), UNKNOWNC("?");

    // Das Befehlswort als Zeichenkette
    private String befehlswort;

    /**
     * Initialisieren mit dem entsprechenden Befehlswort.
     * @param befehlswort das Befehlswort als Zeichenkette
     */
    Befehlswort(String befehlswort)
    {
        this.befehlswort = befehlswort;
    }

    /**
     * @return das Befehlswort als Zeichenkette
     */
    public String toString()
    {
        return befehlswort;
    }
}
```

Die wichtigsten Punkte an dieser neuen Version von **Befehlswort** sind:

- Jeder Wert des Typs hat einen Parameterwert – in diesem Fall den Text des Benutzerbefehls, der mit dem Wert verknüpft sein soll.
- Im Unterschied zur Version in Listing 8.9 ist ein Semikolon am Ende der Liste mit den Werten des Typs nötig.
- Die Typdefinition enthält einen Konstruktor. Dieser fasst in seinem Kopf nicht das Schlüsselwort **public**. Die Konstruktoren von Aufzählungstypen sind niemals öffentlich, weil man von ihnen keine Instanzen erzeugt. Der Parameter, der bei jedem Wert angegeben wurde, wird an diesen Konstruktor übergeben.
- Die Typdefinition enthält ein Datenfeld, **befehlswort**. Im Konstruktor wird der Befehlsstring in diesem Datenfeld gespeichert.
- Die Methode **toString** liefert den Text, der auf diese Weise mit einem Wert des Typs verknüpft wurde.

Da der Text der Befehle nun im Typ **Befehlswort** definiert ist, wird in der Klasse **Befehlsworter** in *Zuul-mit-Enums-V2* ein anderer Weg zur Abbildung von Text auf Aufzählungswerte genommen:

```

gueltigeBefehle = new HashMap<String, Befehlswort>();
for(Befehlswort befehl : Befehlswort.values()) {
    if(befehl != Befehlswort.UNKNOWN) {
        gueltigeBefehle.put(befehl.toString(), befehl);
    }
}

```

Jeder Aufzählungstyp definiert eine Methode **values**, die ein Array der Wert-Objekte des Typs zurückliefert. Der Quelltext oben iteriert über dieses Array und ruft jeweils die Methode **toString** auf, um die verknüpfte Zeichenkette abzufragen.

Übung 8.39 Fügen Sie Ihren eigenen Befehl *look* in *Zuul-mit-Enums-V2* ein. Müssen Sie lediglich den Typ **Befehlswort** ändern?

Übung 8.40 Ändern Sie das Wort, das mit dem Befehl **HELP** verknüpft ist. Ist diese Änderung automatisch im Willkommenstext sichtbar, wenn Sie das Spiel starten? Untersuchen Sie in der Methode **willkommenstextAusgeben** der Klasse **Spiel**, wie dies erreicht wurde.

8.14 Entwurfsregeln

Programmieranfängern werden häufig Tipps gegeben wie „Lassen Sie eine Methode nicht zu viel tun“ oder „Packen Sie nicht alles in eine Klasse“. Beide Vorschläge haben ihre Berechtigung, führen aber leicht zu Fragen wie „Wie lang soll eine Methode denn sein?“ oder „Wie lang soll eine Klasse sein?“.

Nach der Diskussion in diesem Kapitel können diese Fragen nun mit Blick auf Kopplung und Kohäsion beantwortet werden. Eine Methode ist zu lang, wenn sie mehr als eine Aufgabe erfüllt. Eine Klasse ist zu komplex, wenn sie mehr als eine logische Einheit modelliert.

Sie merken, dass diese Antworten keine klar definierten Regeln liefern, was genau zu tun ist. Ein Begriff wie eine *einzelne logische Aufgabe* ist immer noch stark interpretierbar und verschiedene Programmierer entscheiden in vielen Situationen unterschiedlich.

Diese Hinweise sind *Richtlinien*, keine in Stein gemeißelten Regeln. Mit diesen Richtlinien im Hinterkopf werden Sie ihre Klassenentwürfe jedoch erheblich verbessern können, sodass Sie komplexere Aufgaben bearbeiten und bessere und spannendere Programme schreiben können.

Die folgenden Übungen sollen als Anregungen verstanden werden, nicht als fest definierte Aufgaben. Das Spiel kann auf viele Arten erweitert werden – implementieren Sie auch eigene Ideen. Sie müssen nicht alle Übungen durchführen, um ein spannendes Spiel zu erstellen; Sie können aber auch mehr oder andere Ideen implementieren.

Übung 8.41 Führen Sie eine Zeitbegrenzung für Ihr Spiel ein. Wenn eine bestimmte Aufgabe nicht rechtzeitig erledigt wird, verliert der Spieler. Eine Begrenzung kann leicht eingebaut werden, indem die Raumwechsel oder die eingegebenen Befehle gezählt werden. Sie müssen keine Echtzeit berücksichtigen.

Übung 8.42 Implementieren Sie irgendwo eine Falltür (oder eine andere Art von Tür, die nur in einer Richtung durchschritten werden kann).

Übung 8.43 Fügen Sie dem Spiel einen *Beamer* hinzu. Ein Beamer ist ein Gerät, das man *laden* und *auslösen* kann. Wenn Sie den Beamer laden, dann registriert er den aktuellen Raum. Wenn Sie den Beamer auslösen, dann transportiert er Sie augenblicklich in den Raum zurück, in dem er geladen wurde. Der Beamer kann entweder zur Standardausrüstung gehören oder der Spieler muss ihn erst finden. Offensichtlich benötigen Sie auch Befehle zum Laden und Auslösen des Beamers.

Übung 8.44 Fügen Sie Ihrem Spiel geschlossene Türen hinzu. Der Spieler muss einen Schlüssel finden (oder anderweitig erwerben), um eine Tür zu öffnen.

Übung 8.45 Fügen Sie einen Teleporter-Raum hinzu. Jedes Mal, wenn ein Spieler diesen Raum betritt, wird er in einen zufällig ausgewählten anderen Raum „gebeamt“. *Hinweis:* Ein guter Entwurf für diese Aufgabe ist nicht leicht. Es kann interessant sein, mit anderen Studenten Entwurfsalternativen zu diskutieren. (Wir werden einige Entwurfsalternativen für diese Aufgabe am Ende von Kapitel 11 diskutieren. Unternehmungslustige oder fortgeschrittene Leser können schon einmal einen Blick darauf werfen.)

Zusammenfassung

In diesem Kapitel haben wir die sogenannten *nichtfunktionalen Aspekte* einer Anwendung diskutiert. Es ging hier nicht primär darum, eine Anwendung eine Aufgabe ausführen zu lassen, sondern darum, dies mit gut entworfenen Klassen zu tun.

Ein guter Klassenentwurf kann einen großen Unterschied ausmachen, wenn eine Anwendung korrigiert, geändert oder erweitert werden muss. Er erlaubt auch oft, dass Teile der Anwendung in anderen Zusammenhängen (beispielsweise in anderen Projekten) wiederverwendet werden und wir auch später noch von ihm profitieren können.

Es gibt zwei Kernkonzepte, mit denen Klassenentwürfe bewertet werden können: Kopplung und Kohäsion. Kopplung bezieht sich auf die Abhängigkeiten zwischen Klassen und Kohäsion auf die Zerlegung in angemessene Einheiten. Einen guten Entwurf zeichnen eine lose Kopplung und eine hohe Kohäsion aus.

Eine gute Methode, um zu einer guten Struktur zu kommen, ist der Entwurf nach Zuständigkeiten. Bei jeder Funktion, die wir einer Anwendung hinzufügen wollen, versuchen wir festzulegen, welche Klasse für welchen Teil der Aufgabe zuständig sein sollte.

Wenn wir ein Programm erweitern, führen wir regelmäßig ein Refactoring durch, um den Entwurf an die geänderten Anforderungen anzupassen und dafür zu sorgen, dass weiterhin alle Klassen und Methoden eine hohe Kohäsion aufweisen und möglichst lose gekoppelt sind.

NEUE BEGRIFFE IN DIESEM KAPITEL

Code-Duplizierung, Kopplung, Kohäsion, Kapselung, Entwurf nach Zuständigkeiten, implizite Kopplung, Refactoring





Zusammenfassung der Konzepte

- **Kopplung** Der Begriff *Kopplung* beschreibt den Grad der Abhängigkeiten zwischen Klassen. Wir streben für ein System eine möglichst *lose Kopplung* an – also ein System, in dem jede Klasse weitgehend unabhängig ist und mit anderen Klassen nur über möglichst schmale, wohldefinierte Schnittstellen kommuniziert.
- **Kohäsion** Der Begriff *Kohäsion* beschreibt, wie gut eine Programmeinheit eine logische Aufgabe oder Einheit abbildet. In einem System mit *hoher Kohäsion* ist jede Programmeinheit (eine Methode, eine Klasse oder ein Modul) verantwortlich für genau eine wohldefinierte Aufgabe oder Einheit. Ein guter Klassenentwurf weist einen hohen Grad an Kohäsion auf.
- **Code-Duplizierung** Code-Duplizierung (ein Quelltextabschnitt erscheint mehr als einmal in einer Anwendung) ist ein Indiz für einen schlechten Entwurf. Sie sollte vermieden werden.
- **Kapselung** Saubere Kapselung reduziert die Kopplung und führt deshalb zu besseren Entwürfen.
- **Entwurf nach Zuständigkeiten** ist ein Entwurfsprozess, bei dem jeder Klasse eine klare Verantwortung zugewiesen wird. Dieser Prozess kann benutzt werden, um festzulegen, welche Klasse für welche Funktionen in einer Anwendung zuständig sein soll.
- **Änderungen lokal halten** Eines der Hauptziele eines guten Klassenentwurfs lautet: Änderungen lokal halten; eine Änderung an einer Klasse sollte einen möglichst geringen Einfluss auf andere Klassen haben.
- **Kohäsion von Methoden** Eine Methode mit hoher Kohäsion ist verantwortlich für genau eine wohldefinierte Aufgabe.
- **Kohäsion von Klassen** Eine Klasse mit hoher Kohäsion repräsentiert genau eine wohldefinierte Einheit.
- **Refactoring** Refactoring ist eine Aktivität, bei der ein bestehender Entwurf restrukturiert wird, um die Qualität des Klassenentwurfs in Hinsicht auf vorzunehmende Änderungen und Erweiterungen zu erhalten.
- Eine **switch-Anweisung** wählt aus verschiedenen möglichen Anweisungsfolgen eine Folge zur Ausführung aus.

Übung 8.46 Zusatzaufgabe. In der Methode `verarbeiteBefehl` in der Klasse `Spiel` gibt es eine `switch`-Anweisung (oder eine Sequenz von `if`-Anweisungen), mit denen der richtige Befehl für ein erkanntes Befehlsword ermittelt wird. Das ist kein sehr guter Entwurf, da wir jedes Mal, wenn ein neuer Befehl hinzugefügt wird, hier eine zusätzliche Fallunterscheidung implementieren müssen. Können Sie diesen Entwurf verbessern? Entwerfen Sie Klassen, mit denen die Behandlung von Befehlen modularer wird und durch die Befehle leichter hinzugefügt werden können. Implementieren Sie diesen Entwurf. Testen Sie.

Übung 8.47 Fügen Sie dem Spiel Personen hinzu. Personen sind Gegenständen sehr ähnlich, aber sie können reden. Sie sprechen einen Text, wenn ein Spieler sie das erste Mal trifft, und sie können ihm Hinweise geben, wenn dieser ihnen den richtigen Gegenstand gibt.

Übung 8.48 Führen Sie bewegliche Personen ein. Diese sind wie andere Personen auch, bewegen sich jedoch mit jedem eingegebenen Befehl in einen angrenzenden Raum.

Übung 8.49 Fügen Sie Ihrem Projekt eine Klasse `Hauptspiel` hinzu. Definieren Sie darin nur eine `main`-Methode, die ein `Spiel`-Objekt erzeugt und dessen `spielen`-Methode aufruft.



KAPITEL

9

Fehler vermeiden

Lernziele

Zentrale Konzepte in diesem Kapitel: Testen, Modultests, Fehler beseitigen, Tests automatisieren

Java-Konstrukte in diesem Kapitel: (In diesem Kapitel werden keine neuen Java-Konstrukte vorgestellt.)

9.1 Einführung

Wenn Sie die vorherigen Kapitel gelesen und die vorgeschlagenen Übungen implementiert haben, dann haben Sie inzwischen etliche Klassen geschrieben. Möglicherweise haben Sie dabei festgestellt, dass der erste Entwurf einer Klasse selten perfekt ist. Normalerweise ist der Quelltext anfangs nicht vollständig, sodass meist noch einiges getan werden muss, um die Implementierung abzuschließen.

Die Probleme, die Sie dabei haben, werden sich über die Zeit verschieben. Programmieranfänger kämpfen typischerweise mit *Syntaxfehlern*. Syntaxfehler sind die Fehler, die in der Struktur des Quelltextes liegen. Sie sind leicht zu finden, weil der Compiler sie entdeckt und eine Fehlermeldung ausgibt.

Erfahrene Programmierer, die kompliziertere Aufgaben angehen, haben üblicherweise weniger Probleme mit der Syntax einer Sprache. Sie beschäftigen sich stattdessen eher mit *logischen Fehlern*.

Ein logischer Fehler liegt vor, wenn ein Programm ohne Probleme übersetzt und ausgeführt werden kann, aber nicht das gewünschte Ergebnis liefert. Logische Probleme sind sehr viel schwieriger zu finden als Syntaxfehler. Tatsächlich ist oft kaum zu erkennen, dass ein Fehler vorliegt.

Syntaktisch korrekte Programme zu schreiben ist relativ leicht zu erlernen, denn es gibt gute Werkzeuge (wie Compiler), die Syntaxfehler entdecken und anzeigen. Logisch korrekte Programme zu schreiben ist hingegen sehr schwierig für nahezu jedes nichttriviale Problem, und der Beweis für ihre Korrektheit kann für den allgemeinen Fall nicht automatisiert werden. Es ist tatsächlich so schwierig, dass der Großteil der kommerziell vertriebenen Software bekanntermaßen eine signifikante Anzahl von Fehlern enthält.